

JUST_ORIONS Team Notebook

Jashore University of Science and Technology

Contents

1 Setup & Basics 1

1.1 Sublime Build 1

1.2 Basic Setup 1

1.3 PBDS 2

1.4 PBDS Custom Comparator 2

1.5 GP Hash Table 2

1.6 Fast I/O 2

1.7 Debug 2

2 Data Structure 2

2.1 Sparse Table 2

2.2 Sparse Table 2D 2

2.3 Merge Sort Tree (Values ≤ K in Range) 3

2.4 K-th Smallest in Range (Index SegTree) 3

2.5 Persistent Segment Tree 3

2.6 Fenwick Tree 4

2.7 Fenwick 2D 4

2.8 BIT Range Update & Range Query 4

2.9 Implicit Treap 4

2.10 Mo's Algorithm 5

2.11 DSU 5

2.12 Lazy Segment Tree 5

3 Graph Theory 5

3.1 Graph Condense (Kosaraju SCC) 5

3.2 Articulation Points & Bridges 6

3.3 Hopcroft-Karp (Bipartite Matching) 6

3.4 Dinic's Algorithm (Max Flow) 6

3.5 Biconnected Comp. & Bridge Tree 7

3.6 Binary Lifting (LCA & K-th Ancestor) 7

3.7 Tree Flattening (Euler Tour) 7

3.8 Heavy Light Decomposition (HLD) 7

3.9 Centroid Decomposition 8

3.10 Centroid Decomposition (Ahsan's) 8

3.11 LCA and Distance 8

3.12 DSU on Tree (Sack) 9

3.13 2-SAT 9

3.14 Min Cost Max Flow 9

3.15 Floyd-Warshall 9

3.16 Bellman-Ford / SPFA 10

3.17 Kruskal's MST 10

3.18 Eulerian Path (Directed) 10

3.19 Rerooting DP 10

3.20 Link-Cut Tree 10

4 Number Theory 11

4.1 GCD Properties 11

4.2 Extended Euclid and CRT 11

4.3 Linear Diophantine Equation 11

4.4 SPF, Möbius, Primes in O(N) 11

4.5 Segmented Sieve 11

4.6 Phi (Euler Totient) 12

4.7 Miller-Rabin Primality Test 12

4.8 Pollard Rho (Fast Factorization) 12

4.9 Möbius Function and Inversion 12

4.10 GCD and LCM Properties 12

4.11 Gauss Circle Theorem 12

5 Combinatorics 12

5.1 Burnside's Lemma / Polya Enumeration 12

5.2 Exclusion DP (GCD/LCM counting) 13

5.3 Bishop Placement 13

5.4 Stirling Numbers of the 1st Kind $S_1(n, k)$ 13

5.5 Stirling Numbers of the 2nd Kind $S_2(n, k)$ 13

5.6 Bell Numbers $B(n)$ 13

5.7 Catalan Numbers C_n 13

5.8 Eulerian Numbers $E(n, k)$ 13

5.9 Derangements $D(n)$ 13

5.10 Partition Numbers $P(n, k)$ 13

5.11 Lucas Theorem 13

5.12 Binomial Coefficient Properties 13

5.13 Stars and Bars 13

5.14 Generating Functions 13

5.15 Bernoulli Numbers & Faulhaber's 13

6 Math 13

6.1 Basis Vector (XOR Basis) 13

6.2 Matrix Exponentiation 14

6.3 Linear Combination Checking 14

6.4 Math Formula Cheatsheet 14

6.5 FFT (Fast Fourier Transform) 14

6.6 NTT (Number Theoretic Transform) 15

6.7 Lagrange Interpolation 15

6.8 Digit DP 15

6.9 Algebraic Identities 15

6.10 Inequalities 15

6.11 Probability & Expected Value 15

7 Strings 16

7.1 Trie (Binary) 16

7.2 Booth's Algorithm (Min Lex Rotation) 16

7.3 KMP 16

7.4 Z-Algorithm 16

7.5 Manacher's Algorithm 16

7.6 Suffix Array 17

7.7 Aho-Corasick 17

7.8 String Double Hashing 17

7.9 Dynamic String Hash 17

8 Geometry 18

8.1 Geometry Formulas 18

8.2 Closest Pair of Points 18

8.3 Rectangle Distance 18

8.4 Area Enclosed by Three Circles 18

8.5 Polygon Operations 18

8.6 Precision Handling 19

8.7 Geometry Theorems 19

8.8 Half-Plane Intersection 19

8.9 Max Rectangular Sum in Matrix 20

9 Others 20

9.1 Game Theory & Nim 20

9.2 SOS DP (Sum Over Subsets) 20

9.3 Sum of Digits up to N 20

9.4 Number of Digits up to N 20

9.5 Subset Sum (Bitset) 20

9.6 Bit Hacks 21

9.7 Sprague-Grundy (Impartial Games) 21

9.8 FWHT (XOR, OR, AND Convolution) 21

9.9 Knuth Optimization 21

9.10 Li Chao Tree (CHT) 21

9.11 Gaussian Elimination 22

9.12 Berlekamp-Massey 22

9.13 Wavelet Tree 22

9.14 Rope (GNU) 22

1 Setup & Basics

1.1 Sublime Build

Linux:

```
{
    "shell": true,
    "cmd": "g++ -std=c++17 '${file}' -o
    '${file_path}/${file_base_name}' &&
    '${file_path}/${file_base_name}' < input.txt >
    output.txt",
    "working_dir": "${file_path}",
    "selector": "source.cpp"
}
```

Windows:

```
{
    "cmd": ["g++.exe",
    "-std=c++17", "${file}",
    "-o",
    "${file_base_name}.exe",
    "&&" ,
    "${file_base_name}.exe<input.txt>output.txt"],
    "shell":true,
    "working_dir":"$file_path",
    "selector":"source.cpp"
}
```

1.2 Basic Setup

```
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
#pragma GCC optimize("Ofast")
ios_base::sync_with_stdio(false), cin.tie(NULL),
cout.tie(NULL);
#ifdef ONLINE_JUDGE
freopen("input.txt", "r",
stdin),freopen("output.txt", "w", stdout);
```

```
#endif
auto rd = []() { ll x; cin >> x; return x; };
#define dbg(x) cerr << "[" #x "]" = " << (x) << "\n"
mt19937_64 rnd((unsigned
int)chrono::steady_clock::now().time_since_epoch
().count());
int64_t Random(int64_t l, int64_t r) { return (l +
(rnd() % (r - l + 1))); }
// Usage: Random(l, r) returns uniform int in [l, r]
// Use gp_hash_table<ll, ll, custom_hash> mp for
// O(1) lookups without hash collisions
```

1.3 PBDS

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <class T>
using OS = tree<T, null_type, less<T>,
rb_tree_tag,
tree_order_statistics_node_update>;
// to erase in multiset-> less_equal<T> and
// s.erase(s.find_by_order(s.order_of_key(x)))
// ll idx = s.order_of_key(x)
// if(idx == s.size()) -> no lower_bound
// else lb = *s.find_by_order(idx) // as
// 0-indexing
// idx-1 will give highest value which is strictly
// less than x
// for upper_bound->do the same with (x+1)
```

1.4 PBDS Custom Comparator

```
struct cmp {bool operator() (const int &a, const int
&b){return a <= b;}};
typedef
tree<int, null_type, cmp, rb_tree_tag, tree_order_s
tatistics_node_update> ordered_set;
```

1.5 GP Hash Table

```
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM =
        chrono::steady_clock::now().time_since_epoch()
        .count();
        return splitmix64(x + FIXED_RANDOM);
    }
    size_t operator()(
        const pair<uint64_t, uint64_t>& p) const noexcept {
        uint64_t a =
        splitmix64(p.first + 0x9e3779b97f4a7c15ULL);
        uint64_t b = splitmix64(p.second +
        0x9e3779b97f4a7c15ULL + 0xc6a4a7935bd1e995ULL);
```

```
        return (size_t)(a ^ (b << 1));
    }
};
gp_hash_table<ll, ll, custom_hash> mp;
```

1.6 Fast I/O

Reads ints only (no strings/floats). Use for large input (10^6+). Redirect using `./a.out < input.txt`.

```
inline char gc() {
    static char buf[1 << 16];
    static size_t bc, be;
    if (bc >= be) {
        buf[0] = 0, bc = 0;
        be = fread(buf, 1, sizeof(buf), stdin);
    }
    return buf[bc++];
}
int readInt() {
    int a, c;
    while ((a = gc()) < 40);
    if (a == '-') return -readInt();
    while ((c = gc()) >= 48) a = a * 10 + c - 48;
    return a - 48;
}
// ~5x faster than cin/scanf
```

1.7 Debug

`dbg(x1,x2,...)` prints line: `x1=value1 || x2=value2 || ...` to `stderr`. Works with integers, strings, pairs, containers (vector, set, map). Disabled with `-DONLINE_JUDGE` (define `ONLINE_JUDGE` for judges).

```
template<class T> void __p(T x) { cerr<<x; }
template<class T, class U> void __p(pair<T,U> x)
{ cerr<<"("<<x.first<<","<<x.second<<")"; }
template<class T> void __p(vector<T> x)
{ cerr<<"{"<<int f=0;for(auto
↪ y:x){if(f)cerr<<","<<__p(y);f=1;}cerr<<"}"; }
template<class T> void __p(set<T> x)
{ cerr<<"{"<<int f=0;for(auto
↪ y:x){if(f)cerr<<","<<__p(y);f=1;}cerr<<"}"; }
template<class T, class U> void __p(map<T,U> x)
{ cerr<<"{"<<int f=0;for(auto [k,v]:x){if(f)cerr<<","<<
↪ __p(k);cerr<<":"<<__p(v);f=1;}cerr<<"}";
↪ }
void __dbg(const char*) {}
template<class T, class... U> void __dbg(const char*s, T
↪ head, U... tail) {
    int i=0;while(s[i]&&s[i]!='(',')')i++;
    cerr.write(s,i)<<"="<<__p(head);
    if constexpr(sizeof...(tail)) cerr<<" ||
↪ ",<<__dbg(s+i+1,tail...);
}
#ifdef ONLINE_JUDGE
#define dbg(...) cerr<<__LINE__<<":
↪ ",<<__dbg(#__VA_ARGS__,__VA_ARGS__),cerr<<"\n"
```

```
#else
#define dbg(...)
#endif
```

2 Data Structure

2.1 Sparse Table

Range query (max/min/GCD). Idempotent — overlapping intervals OK. Build $O(N \log N)$, query $O(1)$. No updates.

```
struct SparseTable {
    SparseTable(vector<int>& data)
    {
        int n = data.size();
        int maxLog = log2(n) + 1;
        table.assign(n, vector<int>(maxLog));
        for (int i = 0; i < n; ++i) {
            table[i][0] = data[i];
        }
        for (int j = 1; (1 << j) <= n; ++j) {
            for (int i = 0; (i + (1 << j) - 1) < n; ++i) {
                table[i][j] = max(table[i][j - 1], table[i +
                (1 << (j - 1))][j - 1]);
            }
        }
    }
    int query(int L, int R) {
        int j = log2(R - L + 1);
        return max(table[L][j], table[R - (1 << j) +
        1][j]);
    }
    vector<vector<int>> table;
};
```

2.2 Sparse Table 2D

Build $O(NM \log N \log M)$, query $O(1)$. Memory intensive — use for small grids only.

```
struct SparseTable2D {
    static const int LG = 10;
    int n, m;
    vector<vector<int>> a;
    vector<int> lg2;
    vector<vector<vector<vector<int>>>> st;
    SparseTable2D(int n, int m) : n(n), m(m) {
        a.assign(n, vector<int>(m, 0));
        lg2.resize(max(n, m) + 1, 0);
        for (int i = 2; i <= max(n, m); i++) lg2[i] =
        lg2[i >> 1] + 1;
        st.assign(n, vector<vector<vector<int>>>>(m,
        vector<vector<int>>(LG, vector<int>(LG,
        0))));
    }
    void set_val(int i, int j, int val) { a[i][j] = val; }
    void build() {
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++)
```

```

    st[i][j][0][0] = a[i][j];
    for (int a = 0; a < LG; a++) {
        for (int b = 0; b < LG; b++) {
            if (a + b == 0) continue;
            for (int i = 0; i + (1 << a) <= n; i++) {
                for (int j = 0; j + (1 << b) <= m; j++) {
                    if (!a)
                        st[i][j][a][b] = max(st[i][j][a][b-1],
                                                st[i][j + (1 << (b - 1))][a][b - 1]);
                    else
                        st[i][j][a][b] = max(st[i][j][a-1][b],
                                                st[i + (1 << (a - 1))][j][a - 1][b]);
                }
            }
        }
    }
}

int query(int x1, int y1, int x2, int y2) {
    int a = lg2[x2 - x1 + 1], b = lg2[y2 - y1 + 1];
    int m1 = max(st[x1][y1][a][b], st[x2 - (1 << a) + 1][y1][a][b]);
    int m2 = max(st[x1][y2 - (1 << b) + 1][a][b],
                  st[x2 - (1 << a) + 1][y2 - (1 << b) + 1][a][b]);
    return max(m1, m2);
}

// st.set_val(i, j, grid[i][j]);
// st.build(); // st.query(0, 0, 1, 2); // max in
// submatrix [(0,0) to (1,2)]

```

2.3 Merge Sort Tree (Values $\leq K$ in Range)

Segment tree where each node stores a sorted list of its range.
 $\text{query}(1, 0, n-1, ql, qr, k) = \#$ of elements $\leq k$ in $[ql, qr]$.
 Build $O(N \log N)$, query $O(\log^2 N)$, space $O(N \log N)$.

```

struct MergeSortTree {
    int n;
    vector<vector<int>> tree;
    MergeSortTree(const vector<int>& arr) {
        n = arr.size();
        tree.resize(4 * n);
        build(1, 0, n - 1, arr);
    }
    void build(int node, int l, int r, const vector<int>&
    ↪ arr) {
        if (l == r) {
            tree[node].push_back(arr[l]);
            return;
        }
        int mid = (l + r) / 2;
        build(2 * node, l, mid, arr);
        build(2 * node + 1, mid + 1, r, arr);
        merge(tree[2 * node].begin(), tree[2 * node].end(),
              tree[2 * node + 1].begin(), tree[2 * node +
    ↪ 1].end(),
              back_inserter(tree[node]));
    }
}

```

```

int query(int node, int l, int r, int ql, int qr, int
    ↪ k) {
    if (l > qr || r < ql) return 0;
    if (l >= ql && r <= qr) {
        // binary search in sorted node list: count <= k
        return upper_bound(tree[node].begin(),
                           tree[node].end(), k) - tree[node].begin();
    }
    int mid = (l + r) / 2;
    return query(2 * node, l, mid, ql, qr, k) +
           query(2 * node + 1, mid + 1, r, ql, qr, k);
}

```

2.4 K-th Smallest in Range (Index SegTree)

Merge sort tree variant. Nodes store sorted *indices* of elements (sorted by value). $\text{query}(0, 0, n-1, l, r, k)$ returns the **original index** of the k -th smallest value in subarray $[l, r]$. Build $O(N \log N)$. Query $O(\log^2 N)$.

```

struct KthSmallest {
    int n;
    vector<vector<int>> seg;
    KthSmallest(const vector<int>& arr) {
        n = arr.size();
        seg.resize(4 * n);
        vector<pair<int, int>> B(n);
        for (int i = 0; i < n; i++) B[i] = {arr[i], i};
        sort(B.begin(), B.end());
        build(0, 0, n - 1, B);
    }
    void build(int node, int l, int r,
    const vector<pair<int, int>>& B) {
        if (l == r) {
            seg[node].push_back(B[l].second);
            return;
        }
        int mid = (l + r) / 2;
        build(2 * node + 1, l, mid, B);
        build(2 * node + 2, mid + 1, r, B);
        merge(seg[2 * node + 1].begin(),
              seg[2 * node + 1].end(),
              seg[2 * node + 2].begin(),
              seg[2 * node + 2].end(),
              back_inserter(seg[node]));
    }
    int query(int node, int st, int end, int l, int r,
    int k) {
        if (st == end) return seg[node][0]; // found k-th
        // p = # of left-subtree indices within index range
        ↪ [l, r]
        int p = upper_bound(seg[2 * node + 1].begin(),
                           seg[2 * node + 1].end(), r)
              - lower_bound(seg[2 * node + 1].begin(),
                           seg[2 * node + 1].end(), l);
        int mid = (st + end) / 2;
        // descend into subtree containing the k-th element
        if (p >= k)

```

```

        return query(2 * node + 1, st, mid, l, r, k);
        else return query(2 * node + 2, mid + 1, end, l,
        ↪ r, k - p);
    }
};

```

2.5 Persistent Segment Tree

K -th smallest in range. Each version = new root.

Usage: $\text{build}() \rightarrow \text{update}(\text{root}_{i-1})$ for each element $\rightarrow \text{query}(\text{root}_{l-1}, \text{root}_r, k)$.

```

struct PST {
    struct Node { int l, r, val; };
    vector<Node> t;
    int T = 0;
    PST(int max_nodes) { t.resize(max_nodes); }
    int build(int b, int e) {
        int cur = T++;
        if (b == e) return cur;
        int m = (b + e) / 2;
        t[cur].l = build(b, m);
        t[cur].r = build(m + 1, e);
        return cur;
    }
    int update(int pre, int b, int e, int i, int v) {
        int cur = T++; // clone previous version
        t[cur] = t[pre]; // copy-on-write
        if (b == e) { t[cur].val += v; return cur; }
        int m = (b + e) / 2;
        if (i <= m) t[cur].l = update(t[pre].l, b, m, i, v);
        else t[cur].r = update(t[pre].r, m + 1, e, i, v);
        t[cur].val = t[t[cur].l].val + t[t[cur].r].val;
        return cur;
    }
    // k-th smallest in value range [b,e] between versions
    ↪ pre..cur
    // difference of left subtree counts = elements in
    ↪ [b,m] added in [l,r]
    int query(int pre, int cur, int b, int e, int k) {
        if (b == e) return b; // value b appears >= k times
        int cnt = t[t[cur].l].val - t[t[pre].l].val;
        int m = (b + e) / 2;
        if (cnt >= k)
            return query(t[pre].l, t[cur].l, b, m, k);
        return query(t[pre].r, t[cur].r, m + 1, e,
        ↪ k - cnt);
    }
};

```

Online Range Distinct Elements: Instead of value frequencies, index i stores $+1$, and if $a[i]$ appeared at *prev*, update -1 at *prev*. Then query sum in $[l, r]$ on root_r .

```

int query_distinct(int cur, int b, int e, int l, int r)
    ↪ {
    if (l > e || r < b) return 0;

```

```

if (l <= b && e <= r) return t[cur].val;
int m = (b + e) / 2;
return query_distinct(t[cur].l, b, m, l, r) +
       query_distinct(t[cur].r, m + 1, e, l, r);
}
// root[0] = build(1, n)
// for i=1..n
//   int r = root[i-1]
//   if (last.count(a[i])) r = update(r, 1, n,
↪ last[a[i]], -1)
//   root[i] = update(r, 1, n, i, 1); last[a[i]] = i
// distinct in [L, R] = query_distinct(root[R], 1, n, L,
↪ R)

```

2.6 Fenwick Tree

Point update, prefix/range sum. $O(\log N)$ per op.
 1-indexed internally. Usage: BIT<ll> fenw(n);
 fenw.add(i, val); fenw.sum(l, r);

```

template<typename T>
struct BIT {
    int n;
    vector<T> bit;
    BIT(int n = 0) { init(n); }
    void init(int n_) { n = n_; bit.assign(n + 1, 0); }
    void add(int i, T v) {
        // walk up: i += i & -i moves to next responsible
        ↪ ancestor
        for (; i <= n; i += i & -i) bit[i] += v; }
    T sum(int i) {
        T r = 0;
        // walk down: i -= i & -i strips lowest set bit
        for (; i > 0; i -= i & -i) r += bit[i];
        return r;
    }
    T sum(int l, int r) { return sum(r) - sum(l - 1); }
};

```

Usage: BIT<ll> fenw(5);

2.7 Fenwick 2D

```

struct BIT2D {
    int n, m;
    vector<vector<ll>> bit;
    BIT2D(int n, int m): n(n), m(m) {
        bit.assign(n + 1, vector<ll>(m + 1, 0));
    }
    void add(int x, int y, ll val) {
        for (int i = x; i <= n; i += i & -i)
            for (int j = y; j <= m; j += j & -j)
                bit[i][j] += val;
    }
    ll sum(int x, int y) {
        ll res = 0;
        for (int i = x; i > 0; i -= i & -i)
            for (int j = y; j > 0; j -= j & -j)
                res += bit[i][j];
        return res;
    }
};

```

```

}
ll query(int x1, int y1, int x2, int y2) {
    return sum(x2, y2) - sum(x1 - 1, y2) -
           sum(x2, y1 - 1) + sum(x1 - 1, y1 - 1);
}
};
BIT2D fenw2(4, 4);
fenw2.add(1, 1, 5);
fenw2.add(3, 2, 7);
cout << fenw2.query(1, 1, 3, 3) << "\n";

```

2.8 BIT Range Update & Range Query

Uses 2 BITs trick. Add val to $[l, r]$, query sum of $[l, r]$. Derive from prefix sum identity: $\text{psum}(i) = i \cdot \text{bit1}(i) - \text{bit2}(i)$, where bit1 tracks coefficient v and bit2 tracks the offset $v(l - 1)$ to cancel.

```

using ll = long long;
ll bit1[200005], bit2[200005], n, m;
void update(ll bit[], ll i, ll val) {
    for (; i <= n; i += (i & (-i))) bit[i] += val;
}
// Equivalent to adding val to each index in [l, r].
// bit1 holds plain difference array; bit2 cancels
↪ boundary terms.
void range_update(ll l, ll r, ll val) {
    update(bit1, l, val);
    update(bit1, r + 1, -val);
    update(bit2, l, val * (l - 1)); // offset for left
    ↪ boundary
    update(bit2, r + 1, -(r * val)); // offset for right
    ↪ boundary
}
ll sum(ll bit[], ll i) {
    ll re = 0;
    for (; i > 0; i -= (i & (-i))) re += bit[i];
    return re;
}
// prefix_sum(i) = sum of original values in [1, i] after
↪ range updates
ll prefix_sum(ll i) {
    return sum(bit1, i) * i - sum(bit2, i);
}
ll range_sum(ll l, ll r) {
    return prefix_sum(r) - prefix_sum(l - 1);
}

```

range.update(l,r,val);

range.sum(l,r);

2.9 Implicit Treap

Randomized BST with lazy sum/add/reverse. $O(\log N)$ per op, 0-indexed. Init Node *t=0;

cut(t,l,r) – extract $[l, r]$ from t , return Node* to the range. t becomes the rest. paste(t,p,s) – insert subtree s at position p into t .

ins(t,p,v) – insert value v at position p (i.e.

paste(t,p,new Node(v))). erase(t,p) – remove element at p (i.e. delete cut(t,p,p)). setv(t,p,v) – set element at p to v . rev(t,l,r) – reverse range $[l, r]$. add(t,l,r,v) – add v to every element in $[l, r]$. qry(t,l,r) – return sum of $[l, r]$.

Internals (used by cut/paste, don't call directly): merge(l,r) – merge two treaps (all keys of l precede r). split(t,k,l,r) – split t into $[0, k - 1]$ in l , $[k, sz - 1]$ in r . push(t) – propagate lazy flags to children. pull(t) – recompute size and sum from children.

```

struct Node {
    ll pr, val, sum, lz = 0; int sz; bool rev = 0;
    Node *l = 0, *r = 0;
    Node(ll v) : pr(rnd()), val(v), sum(v), sz(1) {}
};

int sz(Node *t) { return t ? t->sz : 0; }
ll sum(Node *t) { return t ? t->sum : 0; }

void pull(Node *t) {
    if (!t) return;
    t->sz = 1 + sz(t->l) + sz(t->r);
    t->sum = t->val + sum(t->l) + sum(t->r);
}

void push(Node *t) {
    if (!t) return;
    if (t->rev) {
        swap(t->l, t->r);
        if (t->l) t->l->rev ^= 1;
        if (t->r) t->r->rev ^= 1;
        t->rev = 0;
    }
    if (t->lz) {
        t->val += t->lz;
        if (t->l) { t->l->lz += t->lz; t->l->sum += t->lz *
        ↪ t->l->sz; }
        if (t->r) { t->r->lz += t->lz; t->r->sum += t->lz *
        ↪ t->r->sz; }
        t->lz = 0;
    }
}

```

```

Node *merge(Node *l, Node *r) {
    if (!l || !r) return l ? l : r;
    if (l->pr > r->pr) { push(l); l->r = merge(l->r, r);
    ↪ pull(l); return l; }
    else { push(r); r->l = merge(l, r->l);
    ↪ pull(r); return r; }
}

```

```

void split(Node *t, int k, Node *&l, Node *&r) {
    if (!t) { l = r = 0; return; } push(t);
    if (sz(t->l) >= k) { split(t->l, k, l, t->l); r = t;
    ↪ pull(r); }
    else { split(t->r, k - sz(t->l) - 1,
    ↪ t->r, r); l = t; pull(l); }
}

```

```

}

// cut range [l,r] out of t, return the extracted
↳ subtree
Node *cut(Node *t, int l, int r) {
    Node *a, *b, *c;
    split(t, l, a, b); split(b, r - l + 1, b, c);
    t = merge(a, c); return b;
}

// paste subtree s into t at position p
void paste(Node *t, int p, Node *s) {
    Node *a, *b;
    split(t, p, a, b);
    t = merge(merge(a, s), b);
}

// operations built on cut/paste
void ins(Node *t, int p, ll v) { paste(t, p, new
↳ Node(v)); }
void erase(Node *t, int p) { delete cut(t, p, p);
↳ }
void setv(Node *t, int p, ll v) { auto *m = cut(t, p,
↳ p); m->val = m->sum = v; m->lz = m->rev = 0;
↳ paste(t, p, m); }
void rev(Node *t, int l, int r) { auto *m = cut(t, l,
↳ r); m->rev ^= 1; paste(t, l, m); }
void add(Node *t, int l, int r, ll v) { auto *m =
↳ cut(t, l, r); m->lz += v; m->sum += v * sz(m);
↳ paste(t, l, m); }
ll qry(Node *t, int l, int r) { auto *m = cut(t, l,
↳ r); ll res = m->sum; paste(t, l, m); return res; }

```

2.10 Mo's Algorithm

add(pos) and rem(pos) must be implemented per problem. Hilbert order for $O(N\sqrt{Q})$. Sort queries first. Block size $B = \sqrt{N}$. Alternating sort order within even/odd blocks cuts the total right-pointer moves nearly in half in practice.

```

// sort queries: blocks of size B, snake-order within
↳ blocks
sort(cu.begin(), cu.end(), [&](auto &x, auto &y){
    if (x.l / B != y.l / B)
        return x.l / B < y.l / B;
    return (x.l / B) % 2 ? x.r > y.r : x.r < y.r;
});
int R = -1, L = 0; // [L,R] is current active window
while (R < r) add(++R); // expand right first
while (L > l) add(--L); // expand left
while (R > r) rem(R--); // shrink right
while (L < l) rem(L++); // shrink left

```

2.11 DSU

```

struct dsu {
    vector<int> par, rank, sz;
    int cnt;
    dsu(int n) {
        par.resize(n);
        iota(par.begin(), par.end(), 0);
    }

```

```

    rank.resize(n, 0);
    sz.resize(n, 1);
    cnt = n;
}
int find(int x) {
    return x == par[x] ? x : par[x] = find(par[x]);
}
bool unite(int x, int y) {
    x = find(x), y = find(y);
    if (x == y) return false;
    if (rank[x] < rank[y]) swap(x, y);
    par[y] = x, sz[x] += sz[y];
    if (rank[x] == rank[y]) rank[x] += 1;
    --cnt; return true;
}
};

```

2.12 Lazy Segment Tree

Range add/set, range sum/max/min. $O(\log N)$ per op. Requires using ll = long long; in setup.

```

struct LSeg {
    int n;
    vector<ll> sm, mx, mn, ad, st;
    vector<char> hz;
    const ll INF = 1e18;

    LSeg(int n): n(n), sm(4*n), mx(4*n), mn(4*n),
        ad(4*n), st(4*n), hz(4*n) {}

    void pull(int i) {
        sm[i] = sm[i*2] + sm[i*2+1];
        mx[i] = max(mx[i*2], mx[i*2+1]);
        mn[i] = min(mn[i*2], mn[i*2+1]);
    }
    void pset(int i, int s, int e, ll v) {
        hz[i] = 1; st[i] = v; ad[i] = 0;
        sm[i] = v * (e - s + 1); mx[i] = mn[i] = v;
    }
    void padd(int i, int s, int e, ll v) {
        if (hz[i]) st[i] += v; else ad[i] += v;
        sm[i] += v * (e - s + 1); mx[i] += v; mn[i] += v;
    }
    void push(int i, int s, int e) {
        if (s == e) return;
        int m = (s + e) / 2;
        if (hz[i]) { pset(i*2, s, m, st[i]); pset(i*2+1, m+1, e, st[i]); hz[i] = 0; }
        if (ad[i]) { padd(i*2, s, m, ad[i]); padd(i*2+1, m+1, e, ad[i]); ad[i] = 0; }
    }
    void upd(int l, int r, ll v, int t, int i=1, int s=0, int
↳ e=-1) {
        if (e < 0) e = n-1;
        if (r < s || e < l) return;
        if (l <= s && e <= r) { t ? pset(i, s, e, v) : padd(i, s, e, v); }
        ↳ return;
        push(i, s, e); int m = (s + e) / 2;
    }
}

```

```

        upd(l, r, v, t, i*2, s, m); upd(l, r, v, t, i*2+1, m+1, e);
        pull(i);
    }
    array<ll, 3> qry(int l, int r, int i=1, int s=0, int e=-1)
↳ {
        if (e < 0) e = n-1;
        if (r < s || e < l) return {0, -INF, INF};
        if (l <= s && e <= r) return {sm[i], mx[i], mn[i]};
        push(i, s, e); int m = (s + e) / 2;
        auto a = qry(l, r, i*2, s, m), b = qry(l, r, i*2+1, m+1, e);
        return {a[0] + b[0], max(a[1], b[1]), min(a[2], b[2])};
    }
    void set(int l, int r, ll v) { upd(l, r, v, 1); }
    void add(int l, int r, ll v) { upd(l, r, v, 0); }
    // auto [s, mx, mn] = seg.qry(l, r); seg.set(l, r, v);
    ↳ seg.add(l, r, v);
};

```

3 Graph Theory

3.1 Graph Condense (Kosaraju SCC)

Builds condensed DAG of strongly connected components. Two DFS passes: first on original graph, second on reversed graph. comps = number of SCCs. comp[u] = component id of node u. *Key idea*: DFS1 on original records finish-order; DFS2 on reverse assigns SCCs in reverse finish-order (Kosaraju's trick).

```

struct SCC {
    int n, comps;
    vector<vector<int>> adj, rev_adj, scc_adj;
    vector<int> order, comp;
    vector<bool> vis;
    SCC(int _n): n(_n), comps(0), adj(_n), rev_adj(_n),
        comp(_n, -1), vis(_n, false) {}
    void add_edge(int u, int v) {
        adj[u].push_back(v);
        rev_adj[v].push_back(u);
    }
    void dfs1(int u) {
        vis[u] = true;
        for (int v : adj[u]) if (!vis[v]) dfs1(v);
        order.push_back(u);
    }
    void dfs2(int u, int id) {
        comp[u] = id;
        for (int v : rev_adj[u])
            if (comp[v] == -1) dfs2(v, id);
    }
    void build() {
        for (int i = 0; i < n; i++) if (!vis[i]) dfs1(i);
        reverse(order.begin(), order.end());
        for (int u : order) {
            if (comp[u] == -1) dfs2(u, comps++);
        }
    }
}

```

```

scc_adj.resize(comps);
for (int u = 0; u < n; u++) {
    for (int v : adj[u]) {
        if (comp[u] != comp[v])
            scc_adj[comp[u]].push_back(comp[v]);
    }
}
};

```

3.2 Articulation Points & Bridges

Finds cut vertices and cut edges in $O(N + M)$.

Root is articulation if it has ≥ 2 children in DFS tree.

Edge (u,v) is a bridge if $\text{low}[v] > \text{tin}[u]$.

Key idea: $\text{low}[u]$ = earliest DFS entry time reachable from subtree of u via at most one back-edge. If a child v cannot reach above u, u is an articulation point.

```

struct Articulation {
    int n, timer;
    vector<vector<int>> adj;
    vector<int> tin, low;
    vector<bool> is_art;
    vector<pair<int, int>> bridges;
    Articulation(int n) : n(n), timer(0), adj(n),
        tin(n, -1), low(n, -1), is_art(n, false) {}
    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void dfs(int u, int p = -1) {
        tin[u] = low[u] = timer++;
        int children = 0;
        for (int v : adj[u]) {
            if (v == p) continue;
            if (tin[v] != -1) {
                low[u] = min(low[u], tin[v]);
            } else {
                children++;
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] >= tin[u] && p != -1)
                    is_art[u] = true;
                if (low[v] > tin[u])
                    bridges.push_back({min(u,v), max(u,v)});
            }
        }
        if (p == -1 && children > 1) is_art[u] = true;
    }
    void build() {
        for (int i = 0; i < n; i++)
            if (tin[i] == -1) dfs(i);
    }
};

```

3.3 Hopcroft-Karp (Bipartite Matching)

Maximum bipartite matching. $O(E\sqrt{V})$.

Left side size = n, right side size = m.

How it works: BFS builds layers of unmatched nodes, then DFS finds augmenting paths simultaneously (multi-augmenting). Each phase increases matching size by at least 1; at most $O(\sqrt{V})$ phases.

$\text{add_edge}(u, v)$ — add edge from left u to right v.

$\text{max_matching}()$ — returns the size of max matching.

$\text{matching_edges}()$ — returns pairs (left_node, right_node).

```

struct HopcroftKarp {
    int n, m;
    vector<vector<int>> g;
    vector<int> dist, pairU, pairV;
    static const int INF = 1e9;
    HopcroftKarp(int n_ = 0, int m_ = 0) {
        n = n_; m = m_;
        g.assign(n, {});
        pairU.assign(n, -1);
        pairV.assign(m, -1);
        dist.assign(n, 0);
    }
    void add_edge(int u, int v) { g[u].push_back(v); }
    bool bfs() {
        queue<int> q;
        for (int u = 0; u < n; u++) {
            if (pairU[u] == -1) {
                dist[u] = 0; q.push(u);
            } else dist[u] = INF;
        }
        bool foundAug = false;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : g[u]) {
                int u2 = pairV[v];
                if (u2 == -1) foundAug = true;
                else if (dist[u2] == INF) {
                    dist[u2] = dist[u] + 1; q.push(u2);
                }
            }
        }
        return foundAug;
    }
    bool dfs(int u) {
        for (int v : g[u]) {
            int u2 = pairV[v];
            if (u2 == -1 || (dist[u2] == dist[u] + 1
                && dfs(u2))) {
                pairU[u] = v; pairV[v] = u; return true;
            }
        }
        dist[u] = INF; return false;
    }
    int max_matching() {
        int matching = 0;

```

```

        while (bfs()) {
            for (int u = 0; u < n; u++)
                if (pairU[u] == -1 && dfs(u)) ++matching;
        }
        return matching;
    }
    vector<pair<int,int>> matching_edges() const {
        vector<pair<int,int>> res;
        for (int v = 0; v < m; v++)
            if (pairV[v] != -1)
                res.emplace_back(pairV[v], v);
        return res;
    }
};

```

3.4 Dinic's Algorithm (Max Flow)

$O(EV^2)$ worst case, $O(E\sqrt{V})$ for unit capacities.

Edge (u,v) with capacity w is added as TWO directed arcs.

e.flow = current flow, e.w = capacity.

Access residual capacity via $\text{e.w} - \text{e.flow}$.

Key idea: BFS builds level graph; DFS sends blocking flow along it. "dead-end" nodes are skipped via $\text{done}[u]$ (current arc trick).

```

const int64_t inf = 1LL << 61;
struct Dinic {
    struct edge { int to, rev;
        int64_t flow, w; };
    int n, s, t;
    vector<int> d, done;
    vector<vector<edge>> g;
    Dinic() {}
    Dinic(int n) { n = n + 10; g.resize(n); }
    void add_edge(int u, int v, int64_t w) {
        edge a = {v, (int)g[v].size(), 0, w};
        edge b = {u, (int)g[u].size(), 0, 0};
        g[u].push_back(a), g[v].push_back(b);
    }
    bool bfs() {
        d.assign(n, -1), d[s] = 0;
        queue<int> Q; Q.push(s);
        while (Q.size()) {
            int u = Q.front(); Q.pop();
            for (auto &e : g[u]) {
                int v = e.to;
                if (!d[v] && e.flow < e.w)
                    d[v] = d[u] + 1, Q.push(v);
            }
        }
        return d[t] != -1;
    }
    int64_t dfs(int u, int64_t flow) {
        if (u == t) return flow;
        for (int &i = done[u]; i < (int)g[u].size(); i++) {
            edge &e = g[u][i];
            if (e.w <= e.flow) continue;
            int v = e.to;

```

```

    if (d[v] == d[u] + 1) {
        int64_t nw = dfs(v, min(flow, e.w - e.flow));
        if (nw > 0) {
            e.flow += nw;
            g[v][e.rev].flow -= nw;
            return nw;
        }
    }
    return 0;
}

int64_t max_flow(int _s, int _t) {
    s = _s, t = _t;
    int64_t flow = 0;
    while (bfs()) {
        done.assign(n, 0);
        while (int64_t nw = dfs(s, inf)) flow += nw;
    }
    return flow;
}
};

```

3.5 Biconnected Comp. & Bridge Tree

Finds 2-edge-connected components (BCCs) — maximal subgraphs with no bridges.

Contract each BCC into a node; original bridges become edges of a tree (bridge tree).

How it works: First DFS finds bridges via $\text{low}[v] > \text{tin}[u]$. Second BFS floods each component avoiding bridges — all nodes reachable without crossing a bridge get the same comp id.

$\text{comp}[u]$ = component id of node u .

tree = the resulting bridge tree (size = comps).

$\text{build_tree}()$ runs both phases. Call after adding all edges.

```

struct BCC {
    int n, timer, comps;
    vector<vector<int>> adj, tree;
    vector<int> tin, low, comp;
    vector<bool> is_bridge, vis;
    vector<pair<int, int>> edges;
    BCC(int _n : n(_n), timer(0), comps(0), adj(_n),
        tin(_n, -1), low(_n, -1), comp(_n, -1),
        vis(_n, false) {}
    void add_edge(int u, int v) {
        adj[u].push_back(edges.size());
        adj[v].push_back(edges.size());
        edges.push_back({u, v});
        is_bridge.push_back(false);
    }
    void dfs(int u, int p = -1) {
        vis[u] = true;
        tin[u] = low[u] = timer++;
        for (int id : adj[u]) {
            int v = edges[id].first ^
                edges[id].second ^ u;

```

```

            if (v == p) continue;
            if (vis[v]) low[u] = min(low[u], tin[v]);
            else {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] > tin[u]) is_bridge[id] = true;
            }
        }
    }
    void build_tree() {
        for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);
        vector<int> q;
        for (int i = 0; i < n; i++) {
            if (comp[i] != -1) continue;
            q.push_back(i); comp[i] = comps++;
            while (!q.empty()) {
                int u = q.back(); q.pop_back();
                for (int id : adj[u]) {
                    int v = edges[id].first ^
                        edges[id].second ^ u;
                    if (comp[v] == -1 && !is_bridge[id]) {
                        comp[v] = comp[u]; q.push_back(v);
                    }
                }
            }
        }
        tree.resize(comps);
        for (int i = 0; i < (int)edges.size(); i++) {
            if (is_bridge[i]) {
                int u = comp[edges[i].first],
                    v = comp[edges[i].second];
                tree[u].push_back(v);
                tree[v].push_back(u);
            }
        }
    }
};

```

3.6 Binary Lifting (LCA & K-th Ancestor)

Preprocess $O(N \log N)$, query $O(\log N)$.

$\text{lift}(u, k)$ returns k -th ancestor (or -1).

$\text{query}(u, v)$ returns LCA. Call $\text{dfs}(\text{root}, -1)$ first.

Key idea: $\text{up}[u][i] = 2^i$ -th ancestor of u . Lift deeper node to same depth, then binary-search highest ancestor where paths diverge.

```

struct LCA {
    int n, LOG;
    vector<vector<int>> adj, up;
    vector<int> depth;
    LCA(int _n : n(_n), LOG(_lg(_n) + 2), adj(_n),
        up(_n, vector<int>(LOG, -1)), depth(_n, 0) {}
    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void dfs(int u, int p = -1, int d = 0) {

```

```

        up[u][0] = p;
        depth[u] = d;
        for (int i = 1; i < LOG; i++) {
            if (up[u][i - 1] != -1)
                up[u][i] = up[up[u][i - 1]][i - 1];
        }
        for (int v : adj[u])
            if (v != p) dfs(v, u, d + 1);
    }
    int lift(int u, int k) {
        for (int i = 0; i < LOG; i++) {
            if ((k >> i) & 1) u = up[u][i];
            if (u == -1) break;
        }
        return u;
    }
    int query(int u, int v) {
        if (depth[u] < depth[v]) swap(u, v);
        u = lift(u, depth[u] - depth[v]);
        if (u == v) return u;
        for (int i = LOG - 1; i >= 0; i--) {
            if (up[u][i] != up[v][i]) {
                u = up[u][i]; v = up[v][i];
            }
        }
        return up[u][0];
    }
};

```

3.7 Tree Flattening (Euler Tour)

Key idea: $\text{in}[u]$ = DFS entry time, $\text{out}[u]$ = DFS exit time. Subtree of u = contiguous segment $[\text{in}[u], \text{out}[u]]$ in flat. Enables any subtree range query on a 1D array (use with Segtree/BIT).

```

struct EulerTour {
    int n, timer;
    vector<vector<int>> adj;
    vector<int> in, out, flat;
    EulerTour(int _n : n(_n), timer(0), adj(_n + 1),
        in(_n + 1), out(_n + 1) {}
    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void dfs(int u, int p = 0) {
        in[u] = ++timer;
        flat.push_back(u);
        for (int v : adj[u])
            if (v != p) dfs(v, u);
        out[u] = timer;
    }
    void build(int root = 1) { dfs(root); }
};

```

3.8 Heavy Light Decomposition (HLD)

Path queries on trees. Decomposes tree into $O(\log N)$ heavy paths.

Call dfs(root, 0) then decompose(root, root).
 Uses BIT internally — swap with Lazy SegTree for range updates.
 query(u, v) returns sum on path u→v.
 addVal(idx, delta) and editVal(idx, value) update node value.

```

struct HLD {
    int n, cur_pos = 1;
    vector<int> arr, parent, size, depth, heavy, head, pos;
    vector<vector<int>> adj;
    BIT<ll> segtree;
    HLD(int n) : n(n), arr(n + 1), parent(n + 1), size(n + 2), depth(n + 2), heavy(n + 2, 0), head(n + 2), pos(n + 2), adj(n + 2), segtree(BIT<ll>(n + 2)) {}
    void add_edge(int u, int v) {
        adj[u].push_back(v), adj[v].push_back(u);
    }
    void dfs(int u, int p) {
        parent[u] = p; size[u] = 1;
        int max_sz = 0;
        for (int v : adj[u]) {
            if (v == p) continue;
            depth[v] = depth[u] + 1;
            dfs(v, u);
            size[u] += size[v];
            if (size[v] > max_sz)
                max_sz = size[v], heavy[u] = v;
        }
    }
    void decompose(int u, int h) {
        head[u] = h; pos[u] = cur_pos++;
        segtree.add(pos[u], arr[u]);
        if (heavy[u]) decompose(heavy[u], h);
        for (int v : adj[u])
            if (v != parent[u] && v != heavy[u])
                decompose(v, v);
    }
    int query(int u, int v) {
        int res = 0;
        while (head[u] != head[v]) {
            if (depth[head[u]] < depth[head[v]])
                swap(u, v);
            res += segtree.sum(pos[head[u]], pos[u]);
            u = parent[head[u]];
        }
        if (depth[u] > depth[v]) swap(u, v);
        res += segtree.sum(pos[u], pos[v]);
        return res;
    }
    void addVal(int idx, int delta) {
        segtree.add(pos[idx], delta);
        arr[idx] += delta;
    }
    void editVal(int idx, int value) {

```

```

        segtree.add(pos[idx], value - arr[idx]);
        arr[idx] = value;
    }
};

```

3.9 Centroid Decomposition

Recursively finds centroids to build $O(\log N)$ depth decomposition tree.
 Useful for path-counting queries (e.g., count paths of length = k).
 Mark centroid removed, recurse on remaining subtrees.

```

vector<vector<int>> adj;
vector<bool> is_removed;
vector<int> subtree_size;
int get_subtree_size(int node, int parent = -1) {
    subtree_size[node] = 1;
    for (int child : adj[node]) {
        if (child == parent || is_removed[child])
            continue;
        subtree_size[node] +=
            get_subtree_size(child, node);
    }
    return subtree_size[node];
}
int get_centroid(int node, int tree_size, int parent = -1) {
    for (int child : adj[node]) {
        if (child == parent || is_removed[child])
            continue;
        if (subtree_size[child] * 2 > tree_size)
            return get_centroid(child, tree_size, node);
    }
    return node;
}
void build_centroid_decomp(int node = 0) {
    int centroid = get_centroid(node, get_subtree_size(node));
    // do something with centroid
    is_removed[centroid] = true;
    for (int child : adj[centroid]) {
        if (is_removed[child]) continue;
        build_centroid_decomp(child);
    }
}

```

3.10 Centroid Decomposition (Ahsan's)

```

int tot = 0;
vector<int> sz(n, 1), removed(n, 0), cenpar(n, -1);
auto dfs_sz = [&](auto &&self, int node, int pr)
-> void {
    tot += 1; sz[node] = 1;
    for (auto &son : g[node]) {
        if (son == pr || removed[son]) continue;
        self(self, son, node);
        sz[node] += sz[son];
    }
}

```

```

};
auto find_cen = [&](auto &&self, int node, int pr)
-> int {
    for (auto &son : g[node]) {
        if (son == pr || removed[son]) continue;
        if (sz[son] > tot / 2)
            return self(self, son, node);
    }
    return node;
};
auto decompose = [&](auto &&self, int node, int pr)
-> void {
    tot = 0; dfs_sz(dfs_sz, node, pr);
    int cen = find_cen(find_cen, node, pr);
    removed[cen] = 1; cenpar[cen] = pr;
    for (auto &son : g[cen]) {
        if (son == pr || removed[son]) continue;
        self(self, son, cen);
    }
};
decompose(decompose, 0, -1);

```

3.11 LCA and Distance

```

vector<vector<int>> t, parents;
vector<int> level;
int mx;
void dfs(int node, int parent, int cLevel) {
    level[node] = cLevel;
    parents[0][node] = parent;
    for (auto &x : t[node])
        if (x != parent) dfs(x, node, cLevel + 1);
}
int kthParent(int a, int k) {
    for (int i = 0; i < mx; i++) {
        if (a == -1) return a;
        if (k & (1 << i)) a = parents[i][a];
    }
    return a;
}
int lca(int a, int b) {
    // equalize depths first
    if (level[a] > level[b])
        a = kthParent(a, level[a] - level[b]);
    else b = kthParent(b, level[b] - level[a]);
    if (a == b) return a; // one is ancestor of the other
    // binary search for last node where paths diverge
    for (int i = mx - 1; i >= 0; i--) {
        if (parents[i][a] != parents[i][b]) {
            a = parents[i][a]; b = parents[i][b];
        }
    }
    return parents[0][a]; // one step above is LCA
}
int distance(int a, int b) {
    return level[a] + level[b] - 2 * level[lca(a, b)];
}

```

3.12 DSU on Tree (Sack)

$O(N \log N)$ offline subtree queries.

Key idea: Process light children first (discard), then heavy child (keep). Re-add all light subtree elements at the end — each element is re-added at most $O(\log N)$ times. The `keep` flag controls whether to clear after processing.

```
int timer = 0;
vector<int> sz, in, out, flat, big;
void dfs_sz(int u, int p) {
    sz[u] = 1, big[u] = 0, in[u] = ++timer,
    flat[timer] = u;
    for (int v : adj[u]) {
        if (v != p) {
            dfs_sz(v, u);
            sz[u] += sz[v];
            if (sz[v] > sz[big[u]]) big[u] = v;
        }
    }
    out[u] = timer;
}
void add(int u, int val) { /* update DS */ }
void dfs_sack(int u, int p, bool keep) {
    for (int v : adj[u])
        if (v != p && v != big[u]) dfs_sack(v, u, 0);
    if (big[u]) dfs_sack(big[u], u, 1);
    for (int v : adj[u])
        if (v != p && v != big[u])
            for (int i = in[v]; i <= out[v]; i++)
                add(flat[i], 1);
    add(u, 1);
    // answer queries for subtree u here
    if (!keep)
        for (int i = in[u]; i <= out[u]; i++)
            add(flat[i], -1);
}
// Init: sz/in/out/flat/big.assign(n+1, 0);
// dfs_sz(1, 0); dfs_sack(1, 0, 1);
```

3.13 2-SAT

Each variable x_i is nodes $2i$ (false) and $2i + 1$ (true).

`either(a,b)` adds clause $(a \vee b)$. `setValue(x)` forces x to true.

`solve()` returns true if satisfiable, assigns `values[i] = 0/1`. Uses iterative Tarjan SCC internally (avoid recursion depth issues).

```
struct TwoSat {
    int N;
    vector<vector<int>>> gr;
    vector<int> values;
    TwoSat(int n = 0) : N(n), gr(2 * n) {}
    int addVar() {
        gr.emplace_back(); gr.emplace_back();
        return N++;
    }
}
```

```
void either(int f, int j) {
    f = max(2 * f, -1 - 2 * f);
    j = max(2 * j, -1 - 2 * j);
    gr[f].push_back(j ^ 1);
    gr[j].push_back(f ^ 1);
}
void setValue(int x) { either(x, x); }
vector<int> val, comp, z;
int time = 0;
int dfs(int i) {
    int low = val[i] = ++time, x;
    z.push_back(i);
    for (int e : gr[i])
        if (!comp[e])
            low = min(low, val[e] ? dfs(e));
    if (low == val[i]) {
        do {
            x = z.back(); z.pop_back();
            comp[x] = low;
            if (values[x >> 1] == -1)
                values[x >> 1] = x & 1;
        } while (x != i);
    }
    return val[i] = low;
}
bool solve() {
    values.assign(N, -1);
    val.assign(2 * N, 0); comp = val;
    for (int i = 0; i < 2 * N; i++)
        if (!comp[i]) dfs(i);
    for (int i = 0; i < N; i++)
        if (comp[2 * i] == comp[2 * i + 1])
            return false;
    return true;
}
// either(a, b) = a OR b ; setValue(x) = x is true
// atMostOne({a,b,c}) = at most one is true
```

3.14 Min Cost Max Flow

Finds min-cost max-flow using potentials (Johnson's). $O(F \cdot E \log V)$.

`addEdge(u, v, cap, cost)` adds edge $u \rightarrow v$ with capacity and cost.

`maxflow(s, t)` returns minimum cost to send max flow from s to t .

Supports negative costs, no negative cycles. Handles up to $\sim 10^3$ edges.

```
const ll INF = 1LL << 62;
struct MCMF {
    struct edge { int v, rev; ll cap, cost; };
    int N;
    vector<vector<edge>>> g;
    vector<ll> dist, pi;
    vector<int> preve, prevv;
    MCMF(int n) : N(n), g(n), dist(n), pi(n),
```

```
preve(n), prevv(n) {}
void addEdge(int u, int v, ll cap, ll cost) {
    g[u].push_back({v, (int)g[v].size(), cap, cost});
    g[v].push_back({u, (int)g[u].size() - 1, 0, -cost});
}
ll maxflow(int s, int t) {
    ll flow = 0, cost = 0;
    while (true) {
        fill(dist.begin(), dist.end(), INF);
        dist[s] = 0;
        priority_queue<pair<ll, int>> pq;
        pq.push({0, s});
        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (-d > dist[u]) continue;
            for (int i = 0; i < (int)g[u].size(); i++) {
                edge &e = g[u][i];
                ll nd = dist[u] + e.cost + pi[u] - pi[e.v];
                if (e.cap > 0 && nd < dist[e.v]) {
                    dist[e.v] = nd;
                    prevv[e.v] = u;
                    preve[e.v] = i;
                    pq.push({-nd, e.v});
                }
            }
        }
        if (dist[t] == INF) break;
        for (int i = 0; i < N; i++)
            pi[i] = min(pi[i] + dist[i], INF);
        ll f = INF;
        for (int u = t; u != s; u = prevv[u])
            f = min(f, g[prevv[u]][preve[u]].cap);
        for (int u = t; u != s; u = prevv[u]) {
            edge &e = g[prevv[u]][preve[u]];
            e.cap -= f;
            g[e.v][e.rev].cap += f;
        }
        flow += f;
        cost += f * pi[t];
    }
    return cost;
};
```

3.15 Floyd-Warshall

All-pairs shortest paths in $O(V^3)$. Handles negative weights. Detects and propagates negative cycles: after main DP, if $m[k][k] < 0$ then all paths through k become $-\infty$ (meaning $-\text{INF}$ here).

```
const ll INF = 1LL << 62;
void floydWarshall(vector<vector<ll>>& m) {
    int n = m.size();
    for (int i = 0; i < n; i++)
        m[i][i] = min(m[i][i], 0LL);
```

```

for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (m[i][k] != INF && m[k][j] != INF)
                m[i][j] = min(m[i][j],
                    max(m[i][k] + m[k][j], -INF));
// detect negative cycles
for (int k = 0; k < n; k++)
    if (m[k][k] < 0)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (m[i][k] != INF && m[k][j] != INF)
                    m[i][j] = -INF;
}

```

3.16 Bellman-Ford / SPFA

Single-source shortest paths with negative edges. $O(VE)$.
On the n -th iteration, if distance still improves $\Rightarrow$ negative cycle. Sets distance to $-\text{INF}$ for all nodes on/reachable from a negative cycle.

```

const ll INF = LLONG_MAX;
struct Edge { int a, b; ll w; };
struct Node { ll dist = INF; int prev = -1; };
void bellmanFord(vector<Node>& nodes,
    vector<Edge>& eds, int s) {
    nodes[s].dist = 0;
    int n = nodes.size();
    for (int i = 0; i < n; i++) {
        bool changed = false;
        for (Edge& e : eds) {
            if (nodes[e.a].dist == INF) continue;
            ll d = nodes[e.a].dist + e.w;
            if (d < nodes[e.b].dist) {
                nodes[e.b].dist =
                    (i < n - 1 ? d : -INF);
                nodes[e.b].prev = e.a;
                changed = true;
            }
        }
        if (!changed) break;
    }
}

```

3.17 Kruskal's MST

Greedy: sort edges by weight, add if it doesn't form a cycle (use DSU). Returns MST edges. $O(E \log E)$. For maximum spanning tree, reverse sort.

```

struct Edge { int u, v; ll w; };
vector<Edge> kruskal(vector<Edge> edges, int n) {
    sort(edges.begin(), edges.end(),
        [](Edge& a, Edge& b){ return a.w < b.w; });
    dsu d(n);
    vector<Edge> mst;
    for (Edge& e : edges)
        if (d.unite(e.u, e.v))

```

```

        mst.push_back(e);
    return mst;
}

```

3.18 Eulerian Path (Directed)

Finds Eulerian path in directed graph. Returns empty vector if none exists.

Requires in-degree == out-degree for all nodes (except start/end, diff = 1).

gr[u] = list of (v, edge_id). nedges = total edges.

```

vector<int> eulerPath(vector<vector<pair<int,int>>>
    & gr, int nedges, int src = 0) {
    int n = gr.size();
    vector<int> deg(n), its(n), used(nedges), ret;
    vector<int> s = {src};
    while (!s.empty()) {
        int x = s.back(), &it = its[x];
        if (it == (int)gr[x].size()) {
            ret.push_back(x); s.pop_back();
        } else {
            auto [y, eid] = gr[x][it++];
            if (!used[eid]) {
                used[eid] = 1;
                s.push_back(y);
            }
        }
    }
    if ((int)ret.size() != nedges + 1) return {};
    reverse(ret.begin(), ret.end());
    return ret;
}

```

3.19 Rerooting DP

Tree DP where answer for each node as root is needed.

Key idea: dfs1 computes dp[u] rooted at an arbitrary root. dfs2 re-roots to each child v by temporarily removing dp[v] from dp[u] then adding dp[u] back into dp[v] before recurring — $O(1)$ re-rooting per edge.

```

void dfs1(int u, int p) {
    dp[u] = 1; // base
    for (int v : adj[u]) {
        if (v == p) continue;
        dfs1(v, u);
        dp[u] += dp[v]; // combine child
    }
}
void dfs2(int u, int p) {
    ans[u] = dp[u]; // store answer for root u
    for (int v : adj[u]) {
        if (v == p) continue;
        // move root from u to v
        dp[u] -= dp[v];
        dp[v] += dp[u];
        dfs2(v, u);
        // restore

```

```

        dp[v] -= dp[u];
        dp[u] += dp[v];
    }
}

```

3.20 Link-Cut Tree

Dynamic tree (link/cut). Supports adding/removing edges and connectivity queries,

all in $O(\log N)$ amortized. 0-indexed nodes.

link(u,v) — add edge. cut(u,v) — remove edge.

connected(u,v) — check if same tree.

Each Node can store subtree data via fix().

```

struct Node {
    Node *p = 0, *pp = 0, *c[2];
    bool flip = 0;
    Node() { c[0] = c[1] = 0; fix(); }
    void fix() {
        if (c[0]) c[0]->p = this;
        if (c[1]) c[1]->p = this;
    }
    void pushFlip() {
        if (!flip) return;
        flip = 0; swap(c[0], c[1]);
        if (c[0]) c[0]->flip ^= 1;
        if (c[1]) c[1]->flip ^= 1;
    }
    int up() { return p ? p->c[1] == this : -1; }
    void rot(int i, int b) {
        int h = i ^ b;
        Node *x = c[i], *y = b == 2 ? x : x->c[h];
        Node *z = b ? y : x;
        if ((y->p = p)) p->c[up()] = y;
        c[i] = z->c[i ^ 1];
        if (b < 2) {
            x->c[h] = y->c[h ^ 1];
            y->c[h ^ 1] = x;
        }
        z->c[i ^ 1] = this;
        fix(); x->fix(); y->fix();
        if (p) p->fix();
        swap(pp, y->pp);
    }
    void splay() {
        for (pushFlip(); p; ) {
            if (p->p) p->p->pushFlip();
            p->pushFlip(); pushFlip();
            int c1 = up(), c2 = p->up();
            if (c2 == -1) p->rot(c1, 2);
            else p->p->rot(c2, c1 != c2);
        }
    }
    Node* first() {
        pushFlip();
        return c[0] ? c[0]->first() : (splay(), this);
    }
}

```

```

}
};
struct LinkCut {
    vector<Node> node;
    LinkCut(int n) : node(n) {}
    void link(int u, int v) {
        makeRoot(&node[u]);
        node[u].pp = &node[v];
    }
    void cut(int u, int v) {
        Node *x = &node[u], *top = &node[v];
        makeRoot(top); x->splay();
        if (x->pp) x->pp = 0;
        else { x->c[0] = top->p = 0; x->fix(); }
    }
    bool connected(int u, int v) {
        Node* nu = access(&node[u])->first();
        return nu == access(&node[v])->first();
    }
    void makeRoot(Node* u) {
        access(u); u->splay();
        if (u->c[0]) {
            u->c[0]->p = 0;
            u->c[0]->flip ^= 1;
            u->c[0]->pp = u;
            u->c[0] = 0; u->fix();
        }
    }
    Node* access(Node* u) {
        u->splay();
        while (Node* pp = u->pp) {
            pp->splay(); u->pp = 0;
            if (pp->c[1])
                pp->c[1]->p = 0, pp->c[1]->pp = pp;
            pp->c[1] = u; pp->fix(); u = pp;
        }
        return u;
    }
};

```

4 Number Theory

4.1 GCD Properties

In particular, recalling that gcd is a positive integer valued function we obtain that $\gcd(a, b \cdot c) = 1$ if and only if $\gcd(a, b) = 1$ and $\gcd(a, c) = 1$. 15. $\gcd(a_1, \dots, a_r) = \gcd(\gcd(a_1, \dots, a_{r-1}), a_r)$ when none = 0 16. $\gcd(a, b) \cdot \text{lcm}(a, b) = |a \cdot b|$, so for any two numbers LCM is always multiple of GCD. 17. In a Cartesian coordinate system, $\gcd(a, b)$ can be interpreted as the number of segments between points with integral coordinates on the straight line segment joining the points (0,0) and (a,b). 18. Two numbers are called relatively prime, or coprime, if their greatest common divisor equals 1. For example, 9 and 28 are relatively prime. 19. Any two consecutive integers will be Co-Prime. Ex: 25,26 is co-prime. 5,6 is co-prime and so on. 20. Two or more odd

consecutive positive integer GCD is also 1.

```

// Inline x86 GCD via idiv | faster than __gcd on old
↳ compilers.
// Avoid for large inputs; prefer std::gcd (C++17)
↳ otherwise.
int gcdFast(int a, int b) {
    int result;
    __asm__ __volatile__(
        "movl %1, %%eax;"
        "movl %2, %%ebx;"
        "CONTD: cmpl $0, %%ebx;"
        "je DONE;"
        "xorl %%edx, %%edx;"
        "idivl %%ebx;"
        "movl %%ebx, %%eax;"
        "movl %%edx, %%ebx;"
        "jmp CONTD;"
        "DONE: movl %%eax, %0;"
        : "=g"(result)
        : "g"(a), "g"(b));
    return result;
}

```

4.2 Extended Euclid and CRT

egcd(a,b,x,y) returns $g = \gcd(a, b)$ with $ax + by = g$.

crt(a1,m1,a2,m2) solves $x \equiv a_1 \pmod{m_1}$, $x \equiv a_2 \pmod{m_2}$.

Returns {x, lcm(m1,m2)} or {0, -1} if no solution.

```

using T = __int128_t;
T egcd(T a, T b, T &x, T &y) {
    T xx = y = 0, yy = x = 1;
    while (b) {
        T q = a / b;
        T t = b; b = a % b; a = t;
        t = xx; xx = x - q * xx; x = t;
        t = yy; yy = y - q * yy; y = t;
    }
    return a;
}

```

pair<T, T> crt(T a1, T m1, T a2, T m2) {

```

    T p, q;
    T g = egcd(m1, m2, p, q);
    if (a1 % g != a2 % g) return {0, -1};
    T m = m1 / g * m2;
    p = (p % m + m) % m;
    q = (q % m + m) % m;
    T x = p * a2 % m * (m1 / g) % m +
        q * a1 % m * (m2 / g) % m;
    return {x % m, m};
}

```

4.3 Linear Diophantine Equation

Solves $Ax + By = C$. Solutions exist iff $g = \gcd(|A|, |B|)$ divides C . General: $x = x_0 + k\frac{B}{g}$, $y = y_0 - k\frac{A}{g}$.

```

11 extgcd(11 a, 11 b, 11 &x, 11 &y) {
    if (b == 0) { x = 1; y = 0; return a; }
}

```

```

11 x1, y1;
11 d = extgcd(b, a % b, x1, y1);
x = y1; y = x1 - y1 * (a / b);
return d;
}
bool find_any_solution(11 a, 11 b, 11 c,
11 &x0, 11 &y0, 11 &g) {
    g = extgcd(abs(a), abs(b), x0, y0);
    if (c % g) return false;
    x0 *= c / g; y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}

```

4.4 SPF, Möbius, Primes in O(N)

```

const int N = 1e6 + 2;
vector<int> spf(N), mob(N), primes;
void build() {
    mob[1] = 1;
    for (int i = 2; i < N; i++) {
        if (spf[i] == 0) { // i is prime
            spf[i] = i, mob[i] = -1, // prime: mu = -1
            primes.push_back(i);
        }
        for (int j = 0; i * primes[j] < N; j++) {
            spf[i * primes[j]] = primes[j]; // smallest prime
            ↳ factor
            if (i % primes[j])
                mob[i * primes[j]] = mob[i] * mob[primes[j]]; //
            ↳ multiplicative
            else mob[i * primes[j]] = 0; // p^2 divides: mu =
            ↳ 0
            if (primes[j] == spf[i]) break; // linear sieve
            ↳ key step
        }
    }
}

```

4.5 Segmented Sieve

Key idea: Sieve small primes $\leq \sqrt{R}$, then strike composites in $[L, R]$ using offsets. Efficient for very large ranges ($L, R \leq 10^{12}$, $R - L \leq 10^7$).

```

vector<char> segmentedSieve(long long L,
long long R) {
    long long lim = sqrt(R);
    vector<char> mark(lim + 1, false);
    vector<long long> primes;
    for (long long i = 2; i <= lim; ++i) {
        if (!mark[i]) {
            primes.emplace_back(i);
            for (long long j = i * i; j <= lim; j += i)
                mark[j] = true;
        }
    }
}

```

```

    }
}
vector<char> isPrime(R - L + 1, true);
for (long long i : primes) {
    ll sm = (L / i) * i;
    if (sm < L) sm += i;
    for (long long j = max(i * i, sm); j <= R; j += i)
        isPrime[j - L] = false;
}
if (L == 1) isPrime[0] = false;
return isPrime;
}

```

4.6 Phi (Euler Totient)

$\phi(n)$ = count of integers in $[1, n]$ coprime to n . Formula:
 $\phi(n) = n \prod_{p|n} (1 - 1/p)$. Computed via trial division.
 phi_1_to_n computes ϕ for all $i \in [1, N]$ using linear sieve:
 $O(N)$.

```

int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0) n /= i;
            result -= result / i;
        }
    }
    if (n > 1) result -= result / n;
    return result;
}

void phi_1_to_n() {
    phi[1] = 1;
    for (int i = 2; i < N; i++) {
        if (phi[i] == 0) {
            primes.push_back(i);
            phi[i] = i - 1;
        }
        for (int p : primes) {
            if (i * p >= N) break;
            if (i % p == 0) {
                phi[i * p] = phi[i] * p; break;
            } else {
                phi[i * p] = phi[i] * (p - 1);
            }
        }
    }
}

```

Sum of numbers $\leq x$ coprime to x : $\sum_{\substack{1 \leq i \leq x \\ \gcd(i, x) = 1}} i = \frac{x \cdot \phi(x)}{2}$.

4.7 Miller-Rabin Primality Test

Deterministic for all 64-bit integers. $O(\log^3 n)$.
 Uses fixed bases [2, 325, 9375, 28178, 450775, 9780504, 1795265022].
 Returns true if n is prime, false if composite.

```

ull pow(ull a, ull t, ull mod) {
    ull r = 1;
    for (; t >= 1, a = (ull)a * a % mod)
        if (t & 1) r = (ull)r * a % mod;
    return r;
}

bool isprime(ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
    s = __builtin_ctzll(n - 1), d = n >> s;
    for (ull a : A) {
        ull p = pow(a % n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
            p = (ull)p * p % n;
        if (p != n - 1 && i != s) return 0;
    }
    return 1;
}

```

4.8 Pollard Rho (Fast Factorization)

Factor any 64-bit integer in $O(n^{1/4})$ expected time.
 Uses Montgomery multiplication for fast modular arithmetic.
 find_factor(n) returns a non-trivial factor (or n if prime).
 Pair with Miller-Rabin: if isprime(n) $\rightarrow$ done, else factor.

```

#define ull uint64_t
struct Montgomery {
    ull n, nr;
    Montgomery(ull n) {
        this->n = n, nr = 1;
        for (int i = 0; i < 6; i++)
            nr *= 2 - n * nr;
    }
    ull reduce(__uint128_t x) const {
        ull q = ull(x) * nr,
        m = ((__uint128_t)q * n) >> 64;
        return (x >> 64) + n - m;
    }
    ull multiply(ull x, ull y) {
        return reduce((__uint128_t)x * y);
    }
};

ull f(ull x, Montgomery& m, ull a) {
    return m.multiply(x, x) + a;
}

ull diff(ull a, ull b) {
    return a > b ? a - b : b - a;
}

const int M = 1024;
const ull SEED = 42;

ull find_factor(ull n, ull x0 = 2, ull a = 1) {
    Montgomery m(n);
    ull x = SEED;
    for (int l = M; l < (1 << 20); l *= 2) {
        ull y = x, p = 1;
        for (int i = 0; i < l; i += M) {
            for (int j = 0; j < M; j++)
                x = f(x, m, a), p = m.multiply(p, diff(x, y));
            if (ull g = gcd(p, n); g != 1) return g;
        }
    }
}

```

```

    }
    return n;
}

```

4.9 Möbius Function and Inversion

The Möbius function $\mu(n)$: $\mu(n) = 1$ if square-free with even # of prime factors; $\mu(n) = -1$ if square-free with odd # of prime factors; $\mu(n) = 0$ if n has a squared prime factor.

Inversion: $g(n) = \sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d)g(n/d)$

• $\sum_{d|n} \mu(d) = [n = 1]$

• $\phi(n) = \sum_{d|n} \mu(d) \frac{n}{d}, \quad \frac{n}{\phi(n)} = \sum_{d|n} \frac{\mu(d)}{d}$

GCD Sums:

• $\sum_{i=1}^n [\gcd(i, n) = k] = \phi\left(\frac{n}{k}\right)$

• $\sum_{i=1}^n \gcd(i, n) = \sum_{d|n} d \cdot \phi\left(\frac{n}{d}\right)$

• $\sum_{i=1}^n \frac{i}{\gcd(i, n)} = \frac{1}{2} \sum_{d|n} d \cdot \phi(d)$

4.10 GCD and LCM Properties

- $\gcd(a, b) = \gcd(b, a \bmod b)$
- If $a|(b \cdot c)$ and $\gcd(a, b) = d$, then $(a/d)|c$
- $\gcd(a, \text{lcm}(b, c)) = \text{lcm}(\gcd(a, b), \gcd(a, c))$
- $\gcd(n^a - 1, n^b - 1) = n^{\gcd(a, b)} - 1$

4.11 Gauss Circle Theorem

Lattice points inside $m^2 + n^2 \leq r^2$:

$$N(r) = 1 + 4 \sum_{i=0}^{\infty} \left(\lfloor \frac{r^2}{4i+1} \rfloor - \lfloor \frac{r^2}{4i+3} \rfloor \right)$$

5 Combinatorics

5.1 Burnside's Lemma / Polya Enumeration

Counts distinct colorings under group actions.

Answer = $\frac{1}{|G|} \sum_{g \in G} (\text{colorings fixed by } g)$.

Code: Rotation group of $n \times m$ grid $\times$ reflection.

See also: count_cycles(p) to compute $k^{c(\sigma)}$.

```

using Permutation = vector<int>;
void operator*=(Permutation& p, Permutation
const& q) {
    Permutation copy = p;
    for (int i = 0; i < p.size(); i++)
        p[i] = copy[q[i]];
}

int count_cycles(Permutation p) {
    int cnt = 0;
    for (int i = 0; i < p.size(); i++) {
        if (p[i] != -1) {
            cnt++;
            for (int j = i; p[j] != -1;) {
                int next = p[j]; p[j] = -1; j = next;
            }
        }
    }
}

```

```

    return cnt;
}
int solve(int n, int m) {
    Permutation p(n*m), p1(n*m), p2(n*m), p3(n*m);
    for (int i = 0; i < n*m; i++) {
        p[i] = i;
        p1[i] = (i % n + 1) % n + i / n * n;
        p2[i] = (i / n + 1) % m * n + i % n;
        p3[i] = (m - 1 - i / n) * n + (n - 1 - i % n);
    }
    set<Permutation> s;
    for (int i1 = 0; i1 < n; i1++) {
        for (int i2 = 0; i2 < m; i2++) {
            for (int i3 = 0; i3 < 2; i3++) {
                s.insert(p); p *= p3;
            }
            p *= p2;
        }
        p *= p1;
    }
    int sum = 0;
    for (Permutation const& p : s)
        sum += 1 << count_cycles(p);
    return sum / s.size();
}

```

5.2 Exclusion DP (GCD/LCM counting)

Count pairs/ k -tuples with given gcd.

Let $d[i]$ = count of elements divisible by i . Then:

Ordered pairs (a, b) , $(a, b) \neq (b, a)$: $f[i] = d[i] \cdot d[i]$

Unordered (with rep): $f[i] = d[i](d[i] + 1)/2$

Unordered (no rep): $f[i] = d[i](d[i] - 1)/2$

Ordered k -tuples: $f[i] = (d[i])^k$

Unordered (no rep): $f[i] = \binom{d[i]}{k}$

Unordered (with rep): $f[i] = \binom{d[i]+k-1}{k}$

5.3 Bishop Placement

Count ways to place K bishops on an $N \times N$ chessboard so no two attack each other.

Bishops on black and white diagonals are independent, so the two color sets are solved separately via DP and combined.

`squares(i)` returns the number of cells on the i -th diagonal. $O(N^2)$.

```

int bishop_placements(int N, int K) {
    if (K > 2 * N - 1) return 0;
    auto squares = [&](int i) {
        if (i > N) return 2 * N - i;
        return i;
    };
    vector<vector<int>> D(N * 2, vector<int>(K + 1));
    for (int i = 0; i < N * 2; ++i) D[i][0] = 1;
    D[1][1] = 1;
    for (int i = 2; i < N * 2; ++i)
        for (int j = 1; j <= K; ++j)
            D[i][j] = D[i-2][j] + D[i-2][j-1] *

```

```

        (squares(i) - j + 1);
    int ans = 0;
    for (int i = 0; i <= K; ++i)
        ans += D[N*2-1][i] * D[N*2-2][K-i];
    return ans;
}

```

5.4 Stirling Numbers of the 1st Kind $S_1(n, k)$

Permutations of n elements with exactly k cycles.

- Recurrence: $S_1(n, k) = S_1(n-1, k-1) + (n-1)S_1(n-1, k)$
- Total sum: $\sum_{k=0}^n S_1(n, k) = n!$
- EGF: $(-\ln(1-x))^k/k!$

5.5 Stirling Numbers of the 2nd Kind $S_2(n, k)$

Partition n elements into k non-empty sets.

- Recurrence: $S_2(n, k) = S_2(n-1, k-1) + k \cdot S_2(n-1, k)$
- Formula: $S_2(n, k) = \frac{1}{k!} \sum_{i=0}^k (-1)^{k-i} \binom{k}{i} i^n$
- EGF: $(\exp(x) - 1)^k/k!$

5.6 Bell Numbers $B(n)$

Total ways to partition a set of n elements.

- $B(n) = \sum_{k=0}^n S_2(n, k)$
- $B(n+1) = \sum_{k=0}^n \binom{n}{k} B(k)$
- EGF: $\exp(\exp(x) - 1)$

5.7 Catalan Numbers C_n

Seq: 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862 ...

- Formula: $C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$
- Recurrence: $C_{n+1} = \sum_{i=0}^n C_i C_{n-i}$
- OGF: $\frac{1-\sqrt{1-4x}}{2x}$
- Applications: Correct bracket seqs, polygon triangulations, non-crossing chords, paths $(0, 0) \rightarrow (n, n)$ below diagonal, full binary trees, Dyck words.

5.8 Eulerian Numbers $E(n, k)$

Permutations with exactly k ascents.

- Recurrence: $E(n, k) = (n-k)E(n-1, k-1) + (k+1)E(n-1, k)$
- Formula: $E(n, k) = \sum_{j=0}^k (-1)^j \binom{n+1}{j} (k+1-j)^n$
- $E(n, k) = E(n, n-1-k)$

5.9 Derangements $D(n)$

Permutations where $\pi(i) \neq i$.

Seq: 1, 0, 1, 2, 9, 44, 265, 1854 ...

- Recurrence: $D(n) = (n-1)(D(n-1) + D(n-2))$
- Formula: $D(n) = n! \sum_{k=0}^n \frac{(-1)^k}{k!} \approx \frac{n!}{e}$
- EGF: $\frac{e^{-x}}{1-x}$
- Code:

```

if (n > 0) D[1] = 0;
if (n > 1) D[2] = 1;
for (int i = 3; i <= n; ++i)
    D[i] = (i - 1) * (D[i - 1] + D[i - 2]);

```

5.10 Partition Numbers $P(n, k)$

Ways to write n as sum of exactly k positive ints.

- Recurrence: $P(n, k) = P(n-1, k-1) + P(n-k, k)$
- Total partitions: $P(n) = \sum_{k=0}^n P(n, k)$

Seq: 1, 1, 2, 3, 5, 7, 11, 15, 22, 30 ...

5.11 Lucas Theorem

Computes $\binom{n}{r} \bmod p$ for prime p , when n, r are large. Breaks n, r into base- p digits and multiplies small binomials.

MOD must be prime and $\leq 10^5$ for precomputed factorials.

Usage: $\binom{10^{18}}{10^9} \bmod 10000000007$ in $O(\log_p n)$ time.

```

long long lucas(long long n, long long r) {
    if (r == 0) return 1;
    return lucas(n / MOD, r / MOD) *
        nCr(n % MOD, r % MOD) % MOD;
}

```

5.12 Binomial Coefficient Properties

- *Symmetry*: $\binom{n}{k} = \binom{n}{n-k}$
- *Pascal (Addition)*: $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$
- *Absorption (Committee)*: $k \binom{n}{k} = n \binom{n-1}{k-1}$
- *Sum (Total Subsets)*: $\sum_{k=0}^n \binom{n}{k} = 2^n$
- *Alternating Sum*: $\sum_{k=0}^n (-1)^k \binom{n}{k} = 0$
- *Even/Odd Sum*: $\sum_{k \text{ even/odd}} \binom{n}{k} = 2^{n-1}$
- *Hockey-Stick*: $\sum_{i=k}^n \binom{i}{k} = \binom{n+1}{k+1}$
- *Sum over Upper Index*: $\sum_{m=0}^n \binom{m}{k} = \binom{n+1}{k+1}$
- *Vandermonde's Convolution*: $\sum_{k=0}^r \binom{m}{k} \binom{n}{r-k} = \binom{m+n}{r}$
- *Sum of Squares*: $\sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}$
- *Weighted*: $\sum_{k=1}^n k \binom{n}{k} = n \cdot 2^{n-1}$
- *Fibonacci*: $\sum_{k \geq 0} \binom{n-k}{k} = F_{n+1}$

5.13 Stars and Bars

$C(n+k-1, k)$

5.14 Generating Functions

- OGF: $A(x) = \sum_{i \geq 0} a_i x^i$
- $xA'(x) \Rightarrow na_n$, $A(x)B(x) \Rightarrow \sum_{i=0}^n a_i b_{n-i}$
- EGF: $A(x) = \sum_{i \geq 0} a_i \frac{x^i}{i!}$
- $A'(x) \Rightarrow a_{n+1}$, $xA(x) \Rightarrow na_n$, $A(x)B(x) \Rightarrow \sum_{i=0}^n \binom{n}{i} a_i b_{n-i}$

5.15 Bernoulli Numbers & Faulhaber's

- Recurrence: $B_n = -\frac{1}{n+1} \sum_{k=0}^{n-1} \binom{n+1}{k} B_k$
- EGF: $\frac{x}{e^x - 1}$
- Sum of powers: $\sum_{k=1}^n k^c = \frac{1}{c+1} \sum_{k=0}^c (-1)^k \binom{c+1}{k} B_k n^{c+1-k}$

6 Math

6.1 Basis Vector (XOR Basis)

Maintains linear basis of vectors mod 2 (XOR space).

`insert(x)` — returns true if x is independent.

maxXor() — max XOR subset of inserted values.
 kth(k) — k-th smallest XOR (1-indexed).
 count_less(x) — count of subsets with XOR < x.
 count_leq(x) — count of subsets with XOR ≤ x.
 can(x) — checks if x is representable.
 reset() — clears the basis for reuse.
 jumpK(val, k) — k-th XOR value ≥ val (rank + k).

```
template<typename T = int, int LG = 31>
struct Basis {
    T basis[LG]; int sz;
    vector<int> who;
    Basis() { reset(); }
    void reset() {
        fill(basis, basis + LG, 0);
        sz = 0; who.clear();
    }
    bool insert(T mask, int idx = -1) {
        for (int i = LG - 1; i >= 0; --i) {
            if (mask >> i & 1) {
                if (basis[i] ^ mask == basis[i])
                    else { basis[i] = mask; ++sz;
                        who.push_back(idx); return true; }
            }
        }
        return false;
    }
    T maxXor() {
        T ans = 0;
        for (int i = LG - 1; i >= 0; --i)
            ans = max(ans, ans ^ basis[i]);
        return ans;
    }
    bool can(T x) {
        for (int i = LG - 1; i >= 0; --i) {
            if (x >> i & 1) {
                if (!basis[i]) return false;
                x ^= basis[i];
            }
        }
        return true;
    }
    T kth(T k) {
        if (k < 1 || k > ((T)1 << sz)) return -1;
        T mask = 0;
        int64_t tot = 1LL << sz;
        for (int i = LG - 1; i >= 0; --i) {
            if (basis[i]) {
                int64_t low = tot / 2;
                if ((low < k && (~mask >> i & 1)) ||
                    (low >= k && (mask >> i & 1)))
                    mask ^= basis[i];
                if (low < k) k -= low;
                tot /= 2;
            }
        }
        return mask;
    }
};
```

```
}
T count_less(T x) {
    if (x < 0) return 0;
    T ret = 0, cnt = ((T)1 << sz), mask = 0;
    for (int i = LG - 1; i >= 0; i--) {
        if (basis[i]) {
            if (x >> i & 1) {
                ret += (cnt >> 1);
                if (~mask >> i & 1) mask ^= basis[i];
            } else {
                if (mask >> i & 1) mask ^= basis[i];
            }
            cnt >>= 1;
        } else {
            if ((x ^ mask) >> i & 1) {
                if (x >> i & 1) return ret + cnt;
                else return ret;
            }
        }
    }
    return ret;
}
T count_leq(T x) { return count_less(x + 1); }
T jumpK(T val, T k) {
    return kth(count_leq(val) + k);
}
};
```

6.2 Matrix Exponentiation

multiply(a,b) multiplies matrices $O(NMP)$.

expo(t,p) computes t^p in $O(N^3 \log p)$.

Skips zero entries in first matrix for speed.

Define mod globally. Use for linear recurrences.

```
vector<vector<int>> multiply(
    vector<vector<int>>& a,
    vector<vector<int>>& b) {
    int n = sz(a), m = sz(a[0]), p = sz(b[0]);
    vector<vector<int>> result(n,
        vector<int>(p, 0));
    for (int i = 0; i < n; i++)
        for (int k = 0; k < m; k++) {
            if (a[i][k] == 0) continue;
            for (int j = 0; j < p; j++)
                result[i][j] = ((result[i][j] +
                    a[i][k] * b[k][j]) % mod + mod) % mod;
        }
    return result;
}
vector<vector<int>> expo(
    vector<vector<int>> t, int p) {
    int n = t.size();
    vector<vector<int>> result(n, vector<int>(n, 0));
    for (int i = 0; i < n; i++) result[i][i] = 1;
    while (p > 0) {
        if (p & 1) result = multiply(result, t);
        t = multiply(t, t);
        p >>= 1;
    }
}
```

```
}
return result;
}
```

6.3 Linear Combination Checking

Given array arr and integer N, check if N is a linear combination of arr's elements: $N = a \cdot x + b \cdot y + c \cdot z + \dots$ Ans: $N \bmod \text{GCD}(\text{all elements}) == 0$.

6.4 Math Formula Cheatsheet

Geometric Series:

$$\bullet \sum_{i=a}^b c^i = \frac{c^{b+1} - c^a}{c - 1} \quad (c \neq 1)$$

$$\bullet \sum_{i=0}^n c^i = \frac{c^{n+1} - 1}{c - 1} \quad (c \neq 1)$$

$$\bullet \sum_{i=0}^{\infty} c^i = \frac{1}{1-c} \quad (|c| < 1)$$

$$\bullet \sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$\bullet \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\bullet \sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4}$$

$$\bullet \sum_{i=1}^n i^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

$$\bullet H_n = \sum_{i=1}^n \frac{1}{i}$$

$$\bullet \sum_{i=1}^n i H_i = \frac{n(n+1)}{2} H_n - \frac{n(n-1)}{4}$$

$$\bullet \sum_{k=0}^n \binom{r+k}{k} = \binom{r+n+1}{n}$$

Harmonic Sums:

Combinatorial Sum:

6.5 FFT (Fast Fourier Transform)

Convolution for integer polynomials (double-based).

multiply(a,b) returns $a * b$ as vector<long long>.

Prefer NTT when modulus supports it (no precision issues).

Complexity $O(N \log N)$ where N is next power of 2.

```
const double PI = acos(-1);
struct base {
    double a, b;
    base(double a = 0, double b = 0) : a(a), b(b) {}
    const base operator+(const base &c) const {
        return base(a + c.a, b + c.b);
    }
    const base operator-(const base &c) const {
        return base(a - c.a, b - c.b);
    }
    const base operator*(const base &c) const {
        return base(a*c.a - b*c.b, a*c.b + b*c.a);
    }
};
void fft(vector<base> &p, bool inv = 0) {
    int n = p.size(), i = 0;
    for (int j = 1; j < n - 1; ++j) {
        for (int k = n >> 1; k > (i ^= k); k >>= 1);
        if (j < i) swap(p[i], p[j]);
    }
    for (int l = 1; (l << 1) <= n; l <<= 1) {
        double ang = 2 * PI / (l << 1);
        base wn(cos(ang), (inv ? 1 : -1) * sin(ang)), w;
        for (int i = 0; i < n; i += l << 1) {
            w = base(1, 0);
            for (int j = i, k = i + l; j < k; ++j, w = w * wn) {
                base t = w * p[j + l];
                p[j + l] = p[j] - t;
            }
        }
    }
}
```

```

        p[j] = p[j] + t;
    }
}
if (inv)
    for (int i = 0; i < n; ++i)
        p[i].a /= n, p[i].b /= n;
}
vector<long long> multiply(vector<long long> &a,
vector<long long> &b) {
    int n = a.size(), m = b.size(), t = n + m - 1,
        sz = 1;
    while (sz < t) sz <= 1;
    vector<base> x(sz), y(sz);
    for (int i = 0; i < sz; ++i) {
        x[i] = i < n ? base(a[i], 0) : base(0, 0);
        y[i] = i < m ? base(b[i], 0) : base(0, 0);
    }
    fft(x), fft(y);
    for (int i = 0; i < sz; ++i) x[i] = x[i] * y[i];
    fft(x, 1);
    vector<long long> ret(sz);
    for (int i = 0; i < sz; ++i)
        ret[i] = (long long)round(x[i].a);
    while (ret.size() > 1 && ret.back() == 0)
        ret.pop_back();
    return ret;
}

```

6.6 NTT (Number Theoretic Transform)

Modular convolution (mod 998244353, primitive root = 62).

Inputs must be in $[0, \text{mod})$ before calling.

conv(a,b) returns polynomial product modulo MOD.

Replace MOD and ROOT for different moduli (mod = $c \cdot 2^k + 1$).

```

const int MOD = 998244353, ROOT = 62;
int modpow(int a, int e) {
    int r = 1;
    while (e) {
        if (e & 1) r = (long long)r * a % MOD;
        a = (long long)a * a % MOD;
        e >>= 1;
    }
    return r;
}
void ntt(vector<int> &a) {
    int n = a.size(), L = 31 - __builtin_clz(n);
    static vector<int> rt(2, 1);
    for (static int k = 2, s = 2; k < n;
k *= 2, s++) {
        rt.resize(n);
        int z[] = {1,
modpow(ROOT, MOD >> s)};
        for (int i = k; i < 2 * k; i++)
            rt[i] = (long long)rt[i / 2] * z[i & 1] % MOD;
    }
    vector<int> rev(n);

```

```

    for (int i = 0; i < n; i++)
        rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    for (int i = 0; i < n; i++)
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k *= 2)
        for (int i = 0; i < n; i += 2 * k)
            for (int j = 0; j < k; j++) {
                int z = (long long)rt[j + k] * a[i + j + k]
                    % MOD;
                a[i + j + k] = a[i + j] - z +
                    (z > a[i + j] ? MOD : 0);
                a[i + j] += a[i + j] + z >= MOD ?
                    z - MOD : z;
            }
    }
    vector<int> conv(const vector<int> &a,
const vector<int> &b) {
    if (a.empty() || b.empty()) return {};
    int s = a.size() + b.size() - 1,
        n = 1 << (32 - __builtin_clz(s));
    int inv = modpow(n, MOD - 2);
    vector<int> L(a), R(b), out(n);
    L.resize(n); R.resize(n);
    ntt(L); ntt(R);
    for (int i = 0; i < n; i++)
        out[(n - i) & (n - 1)] =
            (long long)L[i] * R[i] % MOD * inv % MOD;
    ntt(out);
    return {out.begin(), out.begin() + s};
}

```

6.7 Lagrange Interpolation

Find $f(k)$ given $f(0), f(1), \dots, f(n)$. $f(k) = \sum_{i=0}^n f(i) \cdot$

$$\frac{\prod_{j \neq i} (k-j)}{\prod_{j \neq i} (i-j)}$$

```

// Precompute factorials up to n
vector<int> fact(n + 1, 1), invf(n + 1, 1);
for (int i = 1; i <= n; i++)
    fact[i] = (ll)fact[i - 1] * i % MOD;
invf[n] = modpow(fact[n], MOD - 2);
for (int i = n - 1; i >= 0; i--)
    invf[i] = (ll)invf[i + 1] * (i + 1) % MOD;
int lagrange(vector<int> &y, int k) {
    int n = y.size();
    if (k < n) return y[k];
    vector<int> pre(n + 1, 1), suf(n + 1, 1);
    for (int i = 0; i < n; i++)
        pre[i + 1] = (ll)pre[i] * (k - i) % MOD;
    for (int i = n - 1; i >= 0; i--)
        suf[i] = (ll)suf[i + 1] * (k - i) % MOD;
    int ans = 0;
    for (int i = 0; i < n; i++) {
        int num = (ll)pre[i] * suf[i + 1] % MOD;
        int den = (ll)fact[i] * fact[n - 1 - i] % MOD;
        if ((n - 1 - i) & 1) den = MOD - den;
        ans = (ans + (ll)y[i] * num % MOD *
modpow(den, MOD - 2)) % MOD;
    }
}

```

```

    }
    return ans;
}

```

6.8 Digit DP

Classic: count numbers $\leq N$ with digit sum = S .

State: pos = current digit position, tight = prefix matches N ,

sum = sum of digits so far. Adapt dp[pos][tight][sum] for custom properties.

Reset DP with memset(dp, -1, sizeof(dp)) per query.

```

int dp[20][2][200];
int solve(string& s, int pos, bool tight,
int sum) {
    if (pos == s.size()) return sum == S;
    int &res = dp[pos][tight][sum];
    if (res != -1) return res;
    res = 0;
    int lim = tight ? s[pos] - '0' : 9;
    for (int d = 0; d <= lim; d++)
        res += solve(s, pos + 1,
            tight & (d == lim), sum + d);
    return res;
}
// memset(dp, -1, sizeof(dp));
// solve(to_string(N), 0, 1, 0);

```

6.9 Algebraic Identities

- Squares: $(a \pm b)^2 = a^2 \pm 2ab + b^2 \quad | \quad a^2 - b^2 = (a + b)(a - b)$
- Cubes: $(a \pm b)^3 = a^3 \pm 3a^2b + 3ab^2 \pm b^3 \quad | \quad a^3 \pm b^3 = (a \pm b)(a^2 \mp ab + b^2)$
- Trinomial: $(a + b + c)^2 = a^2 + b^2 + c^2 + 2(ab + bc + ca)$
- 4th Power: $a^4 + b^4 = (a^2 + b^2)^2 - 2a^2b^2$
- n-th Power: $a^n - b^n = (a - b)(a^{n-1} + a^{n-2}b + \dots + b^{n-1})$

6.10 Inequalities

- AM-GM: $\frac{x_1 + \dots + x_n}{n} \geq \sqrt[n]{x_1 \dots x_n}$
- Cauchy-Schwarz: $(\sum a_i^2)(\sum b_i^2) \geq (\sum a_i b_i)^2$
- Triangle: $||a| - |b|| \leq |a \pm b| \leq |a| + |b|$
- Jensen: $f\left(\frac{\sum x_i}{n}\right) \leq \frac{\sum f(x_i)}{n}$ for convex f
- Bernoulli: $(1 + x)^n \geq 1 + nx$ for $n \geq 0, x > -1$
- Chebyshev: $\frac{1}{n} \sum a_i b_i \geq \frac{1}{n} \sum a_i \cdot \frac{1}{n} \sum b_i$ when both sorted similarly
- Hölder: $\sum |x_i y_i| \leq (\sum |x_i|^p)^{1/p} (\sum |y_i|^q)^{1/q}$ where $1/p + 1/q = 1$

6.11 Probability & Expected Value

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad (\text{Bayes})$$

$$E[X] = \sum x \cdot P(X = x)$$

$$E[X + Y] = E[X] + E[Y] \quad (\text{Linearity})$$

$E[X \cdot Y] = E[X] \cdot E[Y]$ if independent

$E[X^2] \geq (E[X])^2$

$\text{Var}(X) = E[X^2] - (E[X])^2$

Markov: $P(X \geq a) \leq \frac{E[X]}{a}$

Chebyshev: $P(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}$

Linearity of Expectation: $E[\sum X_i] = \sum E[X_i]$ always

$\text{Cov}(X, Y) = E[XY] - E[X]E[Y]$

7 Strings

7.1 Trie (Binary)

Binary Trie for integers up to 2^{B-1} . $O(B)$ per op.

$\text{add}(x, v)$ — insert ($v=1$) or erase ($v=-1$)

$\text{count}(x)$ — how many times x inserted

$\text{count_xor_less}(x, k)$ — $\#\{y : (x \wedge y) < k\}$

$\text{count_xor_greater}(x, k)$ — $\#\{y : (x \wedge y) > k\}$

$\text{get_kth}(k)$ — k -th smallest value (1-indexed)

$\text{get_xor_kth}(x, k)$ — k -th smallest $(x \wedge y)$ over y

$\text{count_special}(x)$ — $\#\{y : y \leq (x \wedge y) \leq x\}$

```
struct Trie {
    static const int B = 31;
    vector<array<int, 2>> nxt;
    vector<int> cnt;
    Trie() { nxt.push_back({0, 0}); cnt.push_back(0); }
    void add(int x, int v = 1) {
        int u = 0; cnt[u] += v;
        for (int i = B - 1; i >= 0; --i) {
            int d = (x >> i) & 1;
            if (!nxt[u][d]) {
                nxt[u][d] = nxt.size();
                nxt.push_back({0, 0});
                cnt.push_back(0);
            }
            u = nxt[u][d]; cnt[u] += v;
        }
    }
    int count(int x) {
        int u = 0;
        for (int i = B - 1; i >= 0; --i) {
            int d = (x >> i) & 1;
            if (!nxt[u][d] || !cnt[nxt[u][d]]) return 0;
            u = nxt[u][d];
        }
        return cnt[u];
    }
    int count_xor_less(int x, int k) {
        int u = 0, ans = 0;
        for (int i = B - 1; i >= 0; --i) {
            int d = (x >> i) & 1;
            if ((k >> i) & 1) {
                if (nxt[u][d]) ans += cnt[nxt[u][d]];
                if (!nxt[u][d ^ 1]) break;
                u = nxt[u][d ^ 1];
            } else {
                if (!nxt[u][d]) break;

```

```
                u = nxt[u][d];
            }
        }
        return ans;
    }
    int count_xor_greater(int x, int k) {
        return cnt[0] - count_xor_less(x, k + 1);
    }
    int get_kth(int k) {
        if (cnt[0] < k) return -1;
        int u = 0, ans = 0;
        for (int i = B - 1; i >= 0; --i) {
            int left = nxt[u][0] ? cnt[nxt[u][0]] : 0;
            if (k <= left) u = nxt[u][0];
            else k -= left, u = nxt[u][1], ans |= (1 << i);
        }
        return ans;
    }
    int get_xor_kth(int x, int k) {
        if (cnt[0] < k) return -1;
        int u = 0, ans = 0;
        for (int i = B - 1; i >= 0; --i) {
            int d = (x >> i) & 1;
            int req = nxt[u][d] ? cnt[nxt[u][d]] : 0;
            if (k <= req)
                u = nxt[u][d], ans |= (d << i);
            else k -= req, u = nxt[u][d ^ 1],
                ans |= ((d ^ 1) << i);
        }
        return ans;
    }
    int64_t go(int u, int b, int x, int t1, int t2) {
        if (!u || !t1 || t2 == 2) return 0;
        if ((t1 == 2 && !t2) || b < 0) return cnt[u];
        int d = (x >> b) & 1;
        int64_t res = 0;
        if (d) {
            res += go(nxt[u][1], b - 1, x, t1 == 1 ? 0 : t1,
                ↪ t2 == 1 ? 0 : t2);
            res += go(nxt[u][0], b - 1, x, t1 == 1 ? 2 : t1,
                ↪ t2);
        } else {
            res += go(nxt[u][1], b - 1, x, t1, t2 == 1 ? 2 :
                ↪ t2);
            res += go(nxt[u][0], b - 1, x, t1, t2);
        }
        return res;
    }
    int64_t count_special(int x) { return go(0, B - 1, x,
        ↪ 1, 1); }
};
```

7.2 Booth's Algorithm (Min Lex Rotation)

```
int minLex(string &s) {
    int len = s.size();
    int n = 2 * len, i = 0, j = 1, k = 0;
    int a, b;
    while (i + k < n && j + k < n) {
```

```
        a = (i + k >= len) ? s[i + k - len] : s[i + k];
        b = (j + k >= len) ? s[j + k - len] : s[j + k];
        if (a == b) ++k;
        else if (a > b) {
            i = i + k + 1;
            if (i <= j) i = j + 1;
            k = 0;
        } else {
            j = j + k + 1;
            if (j <= i) j = i + 1;
            k = 0;
        }
    }
    return min(i, j);
}
```

7.3 KMP

```
vector<int> getLps(const string &t) {
    int n = t.size();
    vector<int> pi(n, 0);
    int lps = 0;
    for (int q = 1; q < n; q++) {
        while (lps > 0 && t[lps] != t[q])
            lps = pi[lps - 1];
        if (t[lps] == t[q]) lps += 1;
        pi[q] = lps;
    }
    return pi;
}
```

7.4 Z-Algorithm

```
vector<int> getZ(const string &s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            ↪ z[i]++;
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }
    return z;
}
```

7.5 Manacher's Algorithm

$O(N)$. $p[1][i]$ = odd palindrome radius at i , $p[0][i]$ = even palindrome radius (right center).

```
struct Manacher {
    vector<int> p[2];
    Manacher(string s) {
        int n = s.size();
        p[0].resize(n + 1); // even-length palindromes
        ↪ (right center)
        p[1].resize(n); // odd-length palindromes
        ↪ (center at i)
        for (int z = 0; z < 2; z++) {
```

```

for (int i = 0, l = 0, r = 0; i < n; i++) {
    int t = r - i + !z;           // mirror of i
    ⇨ within [l, r]
    if (i < r) p[z][i] = min(t, p[z][l + t]);
    int L = i - p[z][i], R = i + p[z][i] - !z;
    while (L >= 1 && R + 1 < n && s[L - 1] == s[R +
    ⇨ 1])
        p[z][i]++, L--, R++;
    if (R > r) l = L, r = R;      // update
    ⇨ rightmost palindrome
}
}
bool is_pal(int l, int r) {      // check if
    ⇨ s[l..r] is palindrome
    int mid = (l + r + 1) / 2, len = r - l + 1;
    return 2 * p[len % 2][mid] + len % 2 >= len;
}
};

```

7.6 Suffix Array

$O(N \log N)$ prefix-doubling. Returns SA and LCP.

sa[i] = starting index of i-th smallest suffix.

lcp[i] = LCP of sa[i] and sa[i-1] (i > 0).

Works with string or vector<int>. lim = alphabet size (128 for ASCII).

```

array<vector<int>, 2> get_sa(auto& s, int lim = 128) {
    int n = s.size() + 1, k = 0, a, b;
    vector<int> x(begin(s), end(s) + 1), y(n),
        sa(n), lcp(n), ws(max(n, lim)), rank(n);
    x.back() = 0;
    iota(begin(sa), end(sa), 0);
    for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim
    ⇨ = p) {
        p = j, iota(begin(y), end(y), n - j);
        for (int i = 0; i < n; i++)
            if (sa[i] >= j) y[p++] = sa[i] - j;
        fill(begin(ws), end(ws), 0);
        for (int i = 0; i < n; i++) ws[x[i]]++;
        for (int i = 1; i < lim; i++) ws[i] += ws[i - 1];
        for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
        swap(x, y), p = 1, x[sa[0]] = 0;
        for (int i = 1; i < n; i++)
            a = sa[i - 1], b = sa[i],
            x[b] = (y[a] == y[b] && y[a + j] == y[b + j]) ? p
            ⇨ - 1 : p++;
    }
    for (int i = 1; i < n; i++) rank[sa[i]] = i;
    for (int i = 0, j; i < n - 1; lcp[rank[i++]] = k)
        for (k && k--, j = sa[rank[i] - 1];
            s[i + k] == s[j + k]; k++);
    sa.erase(sa.begin()), lcp.erase(lcp.begin());
    return {sa, lcp};
}

```

7.7 Aho-Corasick

Multiple pattern matching in $O(N + \sum |P| + \text{occ})$.

ins(s) inserts pattern, build_fails() builds fail + output links.

Traverse text via trie[cur][c], follow exit_link[cur] for matches.

is_end[cur] = length of pattern ending at node (0 if none).

```

const int N = 1e6 + 6;
int trie[N][26], fail[N], exit_link[N], is_end[N];
int ptr;

```

```
void init() { ptr = 1; }
```

```

void ins(string &s) {
    int cur = 0;
    for (auto c : s) {
        int d = c - 'a';
        if (!trie[cur][d]) trie[cur][d] = ptr++;
        cur = trie[cur][d];
    }
    is_end[cur] = s.size();
}

```

```

void build_fails() {
    queue<int> q;
    for (int i = 0; i < 26; i++) {
        if (trie[0][i]) q.push(trie[0][i]);
    }
    while (q.size()) {
        int cur = q.front(); q.pop();
        exit_link[cur] = is_end[fail[cur]] > 0 ? fail[cur]
            : exit_link[fail[cur]];
        for (int d = 0; d < 26; d++) {
            if (trie[cur][d]) {
                q.push(trie[cur][d]);
                fail[trie[cur][d]] = trie[fail[cur]][d];
            } else {
                trie[cur][d] = trie[fail[cur]][d];
            }
        }
    }
}

```

7.8 String Double Hashing

Static $O(1)$ substring hash, two moduli. DHash(s) builds prefix hashes. get(l,r) returns pair<ll,ll>. Compare via get(l1,r1)==get(l2,r2).

```

struct DHash {
    ll mod[2]={1000000007,1000000009},
    ⇨ base[2]={911382323,972663749};
    vector<ll> pw[2], h[2];
    DHash(string &s) {
        int n=s.size();
        for(int k=0;k<2;k++) {
            pw[k].resize(n+1); h[k].resize(n+1);

```

```

        for(int i=0;i<n;i++) {
            pw[k][i+1]=pw[k][i]*base[k]%mod[k];
            h[k][i+1]=(h[k][i]*base[k]+s[i])%mod[k];
        }
    }
    pair<ll,ll> get(int l,int r) {
        return {
            (h[0][r+1]-h[0][l]*pw[0][r-l+1]%mod[0]+
            ⇨ mod[0])%mod[0],
            (h[1][r+1]-h[1][l]*pw[1][r-l+1]%mod[1]+
            ⇨ mod[1])%mod[1]
        };
    }
};

```

7.9 Dynamic String Hash

Segment tree with point update ($O(\log N)$), forward/reverse hash, palindrome check. 1-indexed. Set N to max length. Call init_hash(N) once.

```

const int MOD[]={127657753,987654319}, BASE[]={137,277};
int pw[2][N], ipw[2][N];

```

```

int mpow(int a,int n,int m) {
    int r=1;
    while(n) { if(n&1) r=1LL*r*a%m; a=1LL*a*a%m; n>>=1; }
    return r;
}
void init_hash(int n) {
    for(int k=0;k<2;k++) {
        pw[k][0]=ipw[k][0]=1;
        for(int i=1;i<=n;i++)
            ⇨ pw[k][i]=1LL*pw[k][i-1]*BASE[k]%MOD[k];
        int inv=mpow(BASE[k],MOD[k]-2,MOD[k]);
        for(int i=1;i<=n;i++)
            ⇨ ipw[k][i]=1LL*ipw[k][i-1]*inv%MOD[k];
    }
}

```

```

using H = array<int,2>;
H add(H a,H b) { return
    ⇨ {(a[0]+b[0])%MOD[0],(a[1]+b[1])%MOD[1]}; }
H mul(H a,H b) { return {(int)(1LL*a[0]*b[0]%MOD[0]),
    ⇨ (int)(1LL*a[1]*b[1]%MOD[1])}; }
⇨ }

```

```

struct SegHash {
    int n;
    vector<H> fwd,rev;
    SegHash(string s) {
        n=s.size(); s+=" ";
        fwd.resize(4*n); rev.resize(4*n);
        build(1,1,n,s);
    }
    void build(int node,int l,int r,string &s) {
        if(l==r) {

```

```

H v{s[l],s[l]};
fwd[node]=mul(v,H{pw[0][l],pw[1][l]});
rev[node]=mul(v,H{pw[0][n-l+1],pw[1][n-l+1]});
return;
}
int m=(l+r)/2;
build(node*2,l,m,s); build(node*2+1,m+1,r,s);
fwd[node]=add(fwd[node*2],fwd[node*2+1]);
rev[node]=add(rev[node*2],rev[node*2+1]);
}
void upd(int i,char c) { upd(1,1,n,i,c); }
void upd(int node,int l,int r,int i,char c) {
    if(l==r) {
        H v{c,c};
        fwd[node]=mul(v,H{pw[0][l],pw[1][l]});
        rev[node]=mul(v,H{pw[0][n-l+1],pw[1][n-l+1]});
        return;
    }
    int m=(l+r)/2;
    if(i<=m) upd(node*2,l,m,i,c);
    else upd(node*2+1,m+1,r,i,c);
    fwd[node]=add(fwd[node*2],fwd[node*2+1]);
    rev[node]=add(rev[node*2],rev[node*2+1]);
}
H qry(int node,int l,int r,int ql,int qr,vector<H>&t)
↪ {
    if(ql>r||qr<l) return {0,0};
    if(ql<=l&&r<=qr) return t[node];
    int m=(l+r)/2;
    return add(qry(node*2,l,m,ql,qr,t),qry(node*2+1,m+1,
↪ r,ql,qr,t));
}
H get_hash(int l,int r) { return mul(qry(1,1,n,l,r,
↪ fwd),H{ipw[0][l-1],ipw[1][l-1]});
↪ }
H get_rev(int l,int r) { return mul(qry(1,1,n,l,r,
↪ rev),H{ipw[0][n-r],ipw[1][n-r]});
↪ }
bool is_pal(int l,int r) { return
↪ get_hash(l,r)==get_rev(l,r); }
};

```

8 Geometry

8.1 Geometry Formulas

Triangle: Side length a, b, c Area: $\frac{1}{2} \times \text{base} \times \text{height} \ s = \frac{a+b+c}{2}$,
 Area = $\sqrt{s(s-a)(s-b)(s-c)}$ Inradius: $r = \frac{\text{Area}}{s}$ Circum-
 radius: $R = \frac{abc}{4 \times \text{Area}}$ Circles: Arc Length: $L = \theta r$ Sector Area:
 $\text{Area} = \frac{1}{2} r^2 \theta$ Segment Area: $\text{Area} = \frac{1}{2} r^2 (\theta - \sin \theta)$

- **Rhombus:** Area = $\frac{1}{2} \times d_1 \times d_2$
- **Trapezium:** Area = $\frac{1}{2} \times (b_1 + b_2) \times h$
- **Regular Polygon (n sides):** Perimeter = $n \times a$, Area = $\frac{1}{4} n a^2 \cot(\frac{\pi}{n})$
- **Circle:** $C = 2\pi r$, $A = \pi r^2$

- **Ellipse:** $A = \pi ab$

- **Polygon:** Interior sum = $(n - 2) \times 180^\circ$

3D Shapes: **Sphere:** $SA = 4\pi r^2$, $V = \frac{4}{3}\pi r^3$

Cylinder: $SA = 2\pi r(r + h)$, $V = \pi r^2 h$

Cone: $SA = \pi r(r + l)$, $V = \frac{1}{3}\pi r^2 h$

Torus: $SA = 4\pi^2 Rr$, $V = 2\pi^2 Rr^2$

8.2 Closest Pair of Points

Divide-and-conquer, $O(N \log N)$. Splits points by x, recurses on halves, then checks the *strip* of width 2δ around the dividing line. The inner loop runs at most 7 times per point (geometry argument).

```

struct Res { int64_t d; P a, b; };
int64_t sq(int64_t x) { return x * x; }
int64_t dist(P &a, P &b) {
    return sq(a.x - b.x) + sq(a.y - b.y);
}
Res closest_pair(vector<P>& p, vector<P>& py,
vector<P>& tmp, int l, int r) {
    int n = r - l + 1;
    if (n <= 1) return {LLONG_MAX, {}, {}};
    if (n == 2) {
        if (py[l].y > py[l + 1].y)
            swap(py[l], py[l + 1]);
        return {dist(p[l], p[l + 1]), p[l], p[l + 1]};
    }
    int m = (l + r) >> 1, midx = p[m].x;
    Res res_l = closest_pair(p, py, tmp, l, m);
    Res res_r = closest_pair(p, py, tmp, m + 1, r);
    Res res = (res_l.d < res_r.d ? res_l : res_r);
    for (int i = l, j = m + 1, k = 1; k <= r; k++) {
        if (j > r || (i <= m && py[i].y < py[j].y))
            tmp[k] = py[i++];
        else tmp[k] = py[j++];
    }
    for (int i = l; i <= r; i++) py[i] = tmp[i];
    vector<P> stripe;
    for (int i = l; i <= r; i++)
        if (sq(py[i].x - midx) < res.d)
            stripe.push_back(py[i]);
    for (int i = 0; i < (int)stripe.size(); i++) {
        for (int j = i + 1; j < (int)stripe.size(); j++) {
            if (sq(stripe[i].y - stripe[j].y) >= res.d)
                break;
            int64_t d = dist(stripe[i], stripe[j]);
            if (d < res.d)
                res = {d, stripe[i], stripe[j]};
        }
    }
    return res;
}

```

8.3 Rectangle Distance

Distance between the two incircle touch points of the right triangles formed by the diagonal of an $L \times W$ rectangle.

```

long double L, W;
if(!(cin >> L >> W)) return 0;
long double h = sqrt1(L*L + W*W);
long double P = L + W + h;
long double x1 = L*(L + h)/P;
long double y1 = L*W/P;
long double x2 = L*W/P;
long double y2 = W*(W + h)/P;
long double dx = x1 - x2;
long double dy = y1 - y2;
long double D = sqrt1(dx*dx + dy*dy);
cout << fixed << setprecision(6) << (double)D
<< "\n";

```

8.4 Area Enclosed by Three Circles

Black area between three mutually tangent circles of radii r_1, r_2, r_3 (triangle area minus sector areas).

```

long double r1, r2, r3;
cin >> r1 >> r2 >> r3;
long double a = r2 + r3;
long double b = r3 + r1;
long double c = r1 + r2;
long double s = (a + b + c) / 2.0L;
long double T = sqrt1(s*(s - a)*(s - b)*(s - c));
auto angle = [&](long double opp, long double x,
long double y) {
    long double cosv = (x*x + y*y - opp*opp) /
        (2.0L * x * y);
    cosv = min((long double)1.0,
        max((long double)-1.0, cosv));
    return acos1(cosv);
};
long double A = angle(a, b, c);
long double B = angle(b, c, a);
long double C = angle(c, a, b);
long double sectorSum = 0.5L*(r1*r1*A +
r2*r2*B + r3*r3*C);
long double blackArea = T - sectorSum;
cout << fixed << setprecision(6) <<
(double)blackArea << "\n";

```

8.5 Polygon Operations

Key functions available: `dist(a,b)`, `dist_pt_line(a,b,c)`,
`dist_pt_seg(a,b,c)`,
`seg_int(a,b,c,d)`, `line_int(a,b,c,d,res)`, `convexHull(p)`,
`polygon_area(p)`, `is_point_in_polygon(p,q)`.
 P uses int64 coords — use casts for double when needed.

```

const ld EPS = 1e-9;
const ld PI = acos(-1.0);
// sign: -1/0/1 for negative/zero/positive
int sgn(ld x) { return (x > EPS) - (x < -EPS); }

```

```
// 2D point with int64 coords: cross(*), dot(~),
// norm(sq), abs, rotation
struct P {
    using T = int64_t;
    T x, y;
    void read() { cin >> x >> y; }
    P operator - (const P &b) const {
        return P{x - b.x, y - b.y}; }
    P operator + (const P &b) const {
        return P{x + b.x, y + b.y}; }
    bool operator < (const P &b) const {
        return x == b.x ? y < b.y : x < b.x; }
    bool operator == (const P &b) const {
        return x == b.x && y == b.y; }
    T operator * (const P &b) const {
        return x * b.y - y * b.x; }
    T operator ^ (const P &b) const {
        return x * b.x + y * b.y; }
    T dot(P &a, P &b) const {
        return (a - *this) ^ (b - *this); }
    T cross(P &a, P &b) {
        return (a - *this) * (b - *this); }
    ld sq() const { return x * x + y * y; }
    ld abs() const { return sqrtl(sq()); }
    P rot() const { return P(-y, x); } // 90 CCW
    P rot(ld a) const {
        return P(x*cos(a) - y*sin(a),
            x*sin(a) + y*cos(a)); }
};
// true if a, b, c lie on same line
bool collinear(P &a, P &b, P &c) {
    return (b - a) * (c - a) == 0;
}
// true if r lies on closed segment pq
bool on_segment(P &p, P &q, P &r) {
    return collinear(p, q, r) &&
        r.x <= max(p.x, q.x) && r.x >= min(p.x, q.x) &&
        r.y <= max(p.y, q.y) && r.y >= min(p.y, q.y);
}
// Euclidean distance between a and b
ld dist(P a, P b) { return (a - b).abs(); }
// project c onto line ab (returns foot point)
P project(P a, P b, P c) {
    return a + (b - a) * (((c - a) ^ (b - a)) /
        (b - a).sq());
}
// reflect c across line ab
P reflect(P a, P b, P c) {
    return project(a, b, c) * 2 - c;
}
// perpendicular dist from c to line ab
ld dist_pt_line(P a, P b, P c) {
    return fabs1((b - a) * (c - a)) / (b - a).abs();
}
// shortest dist from c to segment ab
ld dist_pt_seg(P a, P b, P c) {
    if (((c - a) ^ (b - a)) <= 0) return (c - a).abs();
    if (((c - b) ^ (a - b)) <= 0) return (c - b).abs();
```

```
    return dist_pt_line(a, b, c);
}
// true if p lies on segment ab (EPS tolerance)
bool on_seg(P a, P b, P p) {
    return !sgn((b - a) * (p - a)) &&
        ((p - a) ^ (p - b)) <= EPS;
}
// line-line intersection (ab x cd), stores in res,
// returns false if parallel
bool line_int(P a, P b, P c, P d, P &res) {
    ld cp = (b - a) * (d - c);
    if (!sgn(cp)) return false;
    res = a + (b - a) * (((d - c) * (c - a)) / cp);
    return true;
}
// true if segment ab intersects segment cd
bool seg_int(P a, P b, P c, P d) {
    auto cp = [&](P x, P y, P z) {
        return sgn((y - x) * (z - x)); };
    if (on_seg(a, b, c) || on_seg(a, b, d) ||
        on_seg(c, d, a) || on_seg(c, d, b))
        return true;
    return cp(a, b, c) != cp(a, b, d) &&
        cp(c, d, a) != cp(c, d, b);
}
// area of polygon (shoelace), vertices in order
long double polygon_area(vector<P> &p) {
    int64_t area = 0, n = p.size();
    for (int i = 0; i < n; ++i)
        area += p[i] * p[(i + 1) % n];
    return abs(area) / 2;
}
// convex hull vertices (CCW, monotone chain)
// Use `<=` for strict hull (no collinear boundary
//   points).
// Use `<` to keep collinear points on hull edges.
vector<P> convexHull(vector<P> p) {
    int n = p.size(), k = 0;
    if (n <= 2) return p;
    vector<P> h(2 * n);
    sort(p.begin(), p.end());
    for (int i = 0; i < n; h[k++] = p[i++])
        while (k >= 2 && h[k - 2].cross(h[k - 1],
            p[i]) <= 0) k--;
    for (int i = n - 2, t = k + 1; i >= 0;
        h[k++] = p[i--])
        while (k >= t && h[k - 2].cross(h[k - 1],
            p[i]) <= 0) k--;
    h.resize(k - 1);
    return h;
}
// point-in-polygon: 1=inside, 2=on boundary, 0=out
// Ray-casting: count crossings of horizontal ray from q
//   to the right.
// Cross-product check ensures correct direction count.
int is_point_in_polygon(const vector<P> &p, P q) {
    int c = 0;
    for (int i = 0, n = p.size(); i < n; ++i) {
```

```
        P a = p[i], b = p[(i + 1) % n];
        if ((b - a) * (q - a) == 0 &&
            min(a.x, b.x) <= q.x &&
            q.x <= max(a.x, b.x) &&
            min(a.y, b.y) <= q.y &&
            q.y <= max(a.y, b.y)) return 2;
        if ((a.y > q.y) != (b.y > q.y) &&
            ((b - a) * (q - a) > 0) == (a.y < b.y))
            c = !c;
    }
    return c;
}
```

8.6 Precision Handling

Helpers for safe float comparison with EPS tolerance.

Use `equal(a,b)` instead of `a == b`, `lessThan(a,b)` instead of `a < b`, etc.

Avoids precision errors from floating-point arithmetic.

```
const double EPS = 1e-9;
bool equal(double a, double b) {
    return abs(a - b) < EPS; }
bool lessThan(double a, double b) {
    return a < b - EPS; }
bool greaterThan(double a, double b) {
    return a > b + EPS; }
```

8.7 Geometry Theorems

Pick's Theorem: For a simple polygon with integer coordinates: $A = I + \frac{B}{2} - 1$, where I = interior lattice points, B = boundary lattice points. **Brahmagupta's Formula:** Area of a cyclic quadrilateral with sides a, b, c, d : $S = \sqrt{(s-a)(s-b)(s-c)(s-d)}$ where $s = \frac{a+b+c+d}{2}$. **Fermat Point:** Point minimizing sum of distances to triangle vertices.

If no angle $> 120^\circ$, sum is $\sqrt{(a^2 + b^2 + c^2 + 4\sqrt{3} \times \text{Area})/2}$.

Simpson's Rule: $\int_a^b f(x)dx \approx \frac{h}{3}[f(x_0) + f(x_n) + 4\sum f(x_{2i+1}) + 2\sum f(x_{2i})]$ **Pythagorean Triples:** $a = 2st, b = s^2 - t^2, c = s^2 + t^2$ for $s > t > 0$, $\gcd(s, t) = 1$, $s \not\equiv t \pmod{2}$. **Euler's Totient Theorem:** If $\gcd(a, n) = 1$, $a^{\phi(n)} \equiv 1 \pmod{n}$. For any a, n , $a^n \equiv a^{n-\phi(n)} \pmod{n}$. For $m \geq \log_2 n$, $a^m \equiv a^{(m \bmod \phi(n)) + \phi(n)} \pmod{n}$.

8.8 Half-Plane Intersection

Finds the convex polygon (or empty region) satisfying all half-planes $a_i x + b_i y \leq c_i$.

Lines are sorted by angle, then a deque maintains the intersection.

`halfPlaneIntersection(lines)` returns vertices of the resulting polygon.

$O(N \log N)$.

```
struct Line {
    double a, b, c; // ax + by <= c
```

```
double ang;
Line(double a = 0, double b = 0, double c = 0)
: a(a), b(b), c(c) {
    ang = atan2(b, a);
}
bool operator < (const Line &o) const {
    return ang < o.ang;
}
};
pair<double,double> intersect(Line &p, Line &q) {
    double d = p.a * q.b - p.b * q.a;
    return {(p.c * q.b - q.c * p.b) / d,
            (p.a * q.c - q.a * p.c) / d};
}
bool ok(Line &a, Line &b, Line &c) {
    auto [x, y] = intersect(a, b);
    return c.a * x + c.b * y < c.c + 1e-9;
}
vector<pair<double,double>>
↳ halfPlaneIntersection(vector<Line> v) {
    sort(v.begin(), v.end());
    deque<Line> dq;
    for (auto &l : v) {
        while (dq.size() >= 2 &&
                !ok(dq[dq.size()-2], dq.back(), l))
            dq.pop_back();
        while (dq.size() >= 2 &&
                !ok(l, dq[0], dq[1]))
            dq.pop_front();
        dq.push_back(l);
    }
    vector<P> res;
    for (int i = 0; i < (int)dq.size(); i++)
        res.push_back(intersect(dq[i],
                                dq[(i+1)%dq.size()]));
    return res;
}
```

8.9 Max Rectangular Sum in Matrix

Key idea: Fix top and bottom rows, collapse 2D into 1D column sums, then apply Kadane's algorithm for max subarray. $O(N^2M)$.

```
int maxRectSum(vector<vector<int>>& a) {
    int n = a.size(), m = a[0].size(), ans = -1e9;
    for (int top = 0; top < n; top++) {
        vector<int> col(m, 0);
        for (int bot = top; bot < n; bot++) {
            for (int j = 0; j < m; j++)
                col[j] += a[bot][j];
            int cur = 0;
            for (int j = 0; j < m; j++) {
                cur = max(col[j], cur + col[j]);
                ans = max(ans, cur);
            }
        }
    }
    return ans;
}
```

```
}
```

9 Others

9.1 Game Theory & Nim

Standard Nim: XOR sum of piles = 0 $\implies$ losing for first player. XOR sum of piles = 0 $\implies$ Second player wins. $\neq 0 \implies$ First player wins. **Subtraction Game:** Can only take up to k stones. XOR sum of $(pile_i \bmod (k+1))$. **Index-k Nim:** Can remove from up to k piles at once. Write sizes in binary. Sum each column. If all column sums $\equiv 0 \pmod{k+1}$, second player wins. **Misère Nim** (Last player to move LOSES): If all piles have size 1, Second wins if N is ODD. Otherwise, play like Standard Nim (XOR sum).

9.2 SOS DP (Sum Over Subsets)

zeta_subset: for each mask, sum over its submasks.

zeta_superset: for each mask, sum over its supermasks.

Complexity $O(B \cdot 2^B)$. Used for gcd/lcm/OR/AND counting.

```
void zeta_subset(vector<int64_t> &dp, int bits) {
    for (int b = 0; b < bits; b++)
        for (int mask = 0; mask < (1 << bits); mask++)
            if (mask >> b & 1)
                dp[mask] += dp[mask ^ (1 << b)];
}
void zeta_superset(vector<int64_t> &dp, int bits) {
    for (int b = 0; b < bits; b++)
        for (int mask = 0; mask < (1 << bits); mask++)
            if (!(mask >> b & 1))
                dp[mask] += dp[mask | (1 << b)];
}
void Solve() {
    ll n; cin >> n;
    vector<ll> arr(n);
    for (auto& x : arr) cin >> x;
    ll N = 1 + __lg(*max_element(arr.begin(),
                                arr.end()));
    vector<ll> sos(1 << N, 0), sos2(1 << N, 0);
    for (auto it : arr) {
        sos[it]++;
        sos2[((1 << N) - 1) ^ it]++;
    }
    for (int i = 0; i < N; ++i)
        for (int mask = 0; mask < (1 << N); ++mask)
            if (mask & (1 << i))
                sos[mask] += sos[mask ^ (1 << i)],
                sos2[mask] += sos2[mask ^ (1 << i)];
    for (auto& it : arr) {
        // count y: x/y == x
        cout << sos[it] << ' ';
        // count y: x&y == x
        cout << sos2[((1 << N) - 1) ^ it] << ' ';
        // count y: x&y != 0
        cout << n - sos2[((1 << N) - 1) ^ it] << ' ';
    }
}
```

9.3 Sum of Digits up to N

Counts total digit sum of all integers $[1, N]$.

Key idea: For each decimal position with factor f , count contribution of each digit value (0-9) to that position across all numbers. $O(\log N)$.

```
uint64_t sumDigitsUpTo(uint64_t n) {
    if (n <= 0) return 0;
    uint64_t sum = 0;
    for (uint64_t f = 1; f <= n; f *= 10) {
        uint64_t hi = n / (f * 10);
        uint64_t cur = (n / f) % 10;
        uint64_t lo = n % f;
        sum += hi * 45LL * f;
        sum += cur * (lo + 1);
        sum += f * (cur * (cur - 1) / 2);
    }
    return sum;
}
```

9.4 Number of Digits up to N

Total count of digits written out in $[1, N]$ (e.g. for $N=12$: 1,2,...,9,1,0,1,1,2 = 15 digits).

Key: Numbers with d digits span a range; count their contribution per digit-length. $O(\log N)$.

```
int64_t get(int64_t n) {
    int64_t base = 1, digits = 1, now = 0;
    while (base * 10 <= n) {
        base *= 10;
        now += (base - (base / 10 == 0 ? 1 :
                                base / 10)) * digits;
        digits += 1;
    }
    n -= (base - 1);
    now += n * digits;
    return now;
}
```

9.5 Subset Sum (Bitset)

Works when sum of values $\leq N$. $O(N\sqrt{N}/64)$.

Uses bitset DP with binary splitting optimization.

`dp.test(k)` is true if sum k can be formed.

```
const int N = 1e6 + 6;
bitset<N> dp;
dp.set(0);
for (auto &[w, c] : comp) {
    for (int i = 0; 1 << i <= c; ++i) {
        dp |= dp << (w * (1 << i));
        c -= 1 << i;
    }
    dp |= dp << (w * c);
}
dp.test(k) // if sum k is possible
```

OR convolution can be done using SOS DP.

9.6 Bit Hacks

```
// --- Basic ops ---
x & -x           // LSB value
x & (x - 1)      // unset LSB
x | (x - 1)      // set trailing 0s to 1
~x & (x + 1)     // isolate lowest 0-bit
x && !(x & (x - 1)) // is power of two?
x >> i & 1       // get bit i
x |= (1 << i)    // set bit i
x &= ~(1 << i)   // clear bit i
x ^= (1 << i)    // toggle bit i
(1 << k) - 1     // mask of k lowest bits
```

```
// --- GCC builtins (suffix ll for 64-bit; undefined for
↳ 0) ---
```

```
__builtin_popcount(x) // # of 1-bits
__builtin_clz(x)      // leading zeros
__builtin_ctz(x)      // trailing zeros
__builtin_parity(x)   // popcount mod 2
```

```
// --- MSB / bit width (32-bit) ---
31 - __builtin_clz(x) // MSB position (0-idx)
32 - __builtin_clz(x) // bit width
1 << (31 - __builtin_clz(x)) // MSB value
```

```
// --- Iterate set bits of x ---
for (int s = x; s; s &= s - 1) {
    int i = __builtin_ctz(s); // current bit index
}
```

```
// --- Iterate submasks of mask m ---
for (int s = m; ; s = (s - 1) & m) {
    if (s == 0) break;
}
```

```
// --- Next number with same popcount ---
int next_popcount(int x) {
    int c = x & -x, r = x + c;
    return r | (((r ^ x) >> 2) / c);
}
```

```
// --- Round up to power of two ---
uint32_t next_pow2(uint32_t n) {
    n--; n |= n>>1; n |= n>>2; n |= n>>4; n |= n>>8; n |=
    ↳ n>>16; return n+1;
}
```

```
// 64-bit: add n |= n>>32
```

```
// --- Gray code ---
int gray = x ^ (x >> 1); // binary -> Gray
// Gray -> binary: g = gray; for (int m = g>>1; m;
↳ m>>=1) g ^= m;
```

9.7 Sprague-Grundy (Impartial Games)

$g(v) = \text{mex}\{g(u) : u \text{ reachable from } v\}$. $g = 0 \implies$ losing, else winning. **XOR** combines sub-games, **mex** computes single state.

```
// DP template (bool[] avoids set overhead)
int g[N]; g[0] = 0;
for (int i = 1; i <= maxN; i++) {
    bool seen[K] = {};
    for (int m : moves) if (i >= m) seen[g[i - m]] = 1;
    int m = 0; while (seen[m]) m++;
    g[i] = m;
}
// Standard Nim:  $g(n) = n$  (XOR pile sizes / first wins
↳ if != 0)
// Subtraction {1..m}:  $g(n) = n \% (m+1)$ 
// Take & Break:  $g(n) = \text{mex}\{g(a) \oplus g(b) \mid a+b=n\}$ 
// Staircase Nim: XOR odd-indexed piles only
// Wythoff: losing pairs = (floor(phi*k),
↳ floor(phi^2*k))
// Pattern: DP 100 terms for periodicity.  $g \gg 1000 \implies$ 
↳ closed form.
// Watch: mex includes 0;  $g(0)=0$ ; add moves irrelevant;
↳ count losing via pairs XOR=0
```

9.8 FWHT (XOR, OR, AND Convolution)

Convolution where $c[k] = \sum_{i \oplus j = k} a[i] \cdot b[j]$.

type: 0=OR, 1=AND, 2=XOR. inv=true for inverse.

modpow(n, MOD-2) for inverse normalization (type 2 only).

```
void fwht(vector<int> &a, bool inv, int type) {
    // type: 0=OR, 1=AND, 2=XOR
    int n = a.size();
    for (int len = 1; len < n; len <= 1)
        for (int i = 0; i < n; i += 2 * len)
            for (int j = 0; j < len; j++) {
                int u = a[i + j], v = a[i + len + j];
                if (type == 2) {
                    a[i + j] = (u + v) % MOD;
                    a[i + len + j] = (u - v + MOD) % MOD;
                } else if (type == 1) {
                    a[i + j] = inv ? (u - v + MOD) % MOD
                                   : (u + v) % MOD;
                } else {
                    a[i + len + j] = inv ? (v - u + MOD) % MOD
                                           : (u + v) % MOD;
                }
            }
    if (inv && type == 2) {
        int inv_n = modpow(n, MOD - 2);
        for (auto &x : a)
            x = (1ll)x * inv_n % MOD;
    }
}
// conv(a, b) = for type=0:  $c[k] = \sum_{i|j=k} a[i]*b[j]$ 
// for type=1:  $c[k] = \sum_{i \& j = k} a[i]*b[j]$ 
// for type=2:  $c[k] = \sum_{i \oplus j = k} a[i]*b[j]$ 
```

9.9 Knuth Optimization

For DP on intervals where optimal split point $p[i][j]$ is monotone in both i and j :

$$a[i][j] = \min_{i < k < j} (a[i][k] + a[k][j]) + f(i, j)$$

Reduces $O(N^3)$ to $O(N^2)$. Applicable when cost f satisfies the quadrangle inequality.

```
// Compute intervals in increasing length
for (int len = 1; len < n; len++) {
    for (int i = 0; i + len <= n; i++) {
        int j = i + len;
        for (int k = p[i][j-1]; k <= p[i+1][j]; k++) {
            ll val = dp[i][k] + dp[k][j] + cost(i, j);
            if (val < dp[i][j]) {
                dp[i][j] = val;
                p[i][j] = k;
            }
        }
    }
}
```

9.10 Li Chao Tree (CHT)

What it does: Maintains a set of lines $y = mx + b$, answers minimum y -value at any query point x .

When to use: DP transitions of the form $dp[j] = \min_i (m_i \cdot x_j + b_i)$ where slopes/intercepts come from earlier states (CHT without monotonicity).

Complexity: $O(\log C)$ per insert/query, C = coordinate range $[-10^9, 10^9]$.

Nodes are allocated *lazily* (only when needed), so memory is $O(N \log C)$.

For *maximum* queries: init with LLONG_MIN and flip comparisons.

```
struct LiChao {
    struct Line { ll m, b; ll get(ll x) { return m * x +
    ↳ b; } };
    struct Node { int l, r; Line line; };
    vector<Node> seg;
    LiChao() { seg.push_back({-1, -1, {0, LLONG_MAX}}); }
    void addLine(Line nw, int idx = 0, ll l = -1e9, ll r
    ↳ = 1e9) {
        ll m = (1 + r) / 2;
        bool left = nw.get(l) < seg[idx].line.get(l);
        bool mid = nw.get(m) < seg[idx].line.get(m);
        if (mid) swap(seg[idx].line, nw);
        if (l == r) return;
        if (left != mid) {
            if (seg[idx].l == -1) {
                seg[idx].l = seg.size();
                seg.push_back({-1, -1, {0, LLONG_MAX}});
            }
            addLine(nw, seg[idx].l, l, m);
        } else {
            if (seg[idx].r == -1) {
                seg[idx].r = seg.size();
                seg.push_back({-1, -1, {0, LLONG_MAX}});
            }
        }
    }
}
```

```

    addLine(nw, seg[idx].r, m + 1, r);
}
}
ll query(ll x, int idx = 0, ll l = -1e9, ll r = 1e9) {
    if (idx == -1) return LLONG_MAX;
    ll m = (l + r) / 2;
    ll res = seg[idx].line.get(x);
    if (l != r) {
        if (x <= m) res = min(res, query(x, seg[idx].l,
            ↪ l, m));
        else res = min(res, query(x, seg[idx].r, m+1, r));
    }
    return res;
}
};
// LiChao lct; lct.addLine({slope, intercept}); ll ans =
↪ lct.query(x);

```

9.11 Gaussian Elimination

Solve $Ax = B$ modulo prime (MOD).

Returns: 0 = no solution, 1 = unique, 2 = infinite solutions.
ans[i] = value of variable i after solving.

Key: Row-reduce to RREF using modular inverse for pivot scaling. Check remaining rows against RHS to detect inconsistency.

```

int gauss(vector<vector<int>>> a,
vector<int>& ans) {
    int n = a.size(), m = a[0].size() - 1;
    vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m && row < n;
        col++) {
        int sel = row;
        for (int i = row; i < n; i++)
            if (a[i][col]) { sel = i; break; }
        if (!a[sel][col]) continue;
        swap(a[sel], a[row]);
        int inv = modpow(a[row][col], MOD - 2);
        for (int j = col; j <= m; j++)
            a[row][j] = (ll)a[row][j] * inv % MOD;
        for (int i = 0; i < n; i++) {
            if (i == row) continue;
            int c = a[i][col];
            for (int j = col; j <= m; j++)
                a[i][j] = (a[i][j] - (ll)c *
                    a[row][j] + MOD) % MOD;
        }
        where[col] = row++; }
    ans.assign(m, 0);
    for (int i = 0; i < m; i++)
        if (where[i] != -1) ans[i] = a[where[i]][m];
    for (int i = 0; i < n; i++) {
        ll sum = 0;
        for (int j = 0; j < m; j++)
            sum = (sum + (ll)ans[j] * a[i][j]) % MOD;
        if (sum != a[i][m]) return 0;
    }
}

```

```

for (int i = 0; i < m; i++)
    if (where[i] == -1) return 2; // infinite
return 1; // unique
}

```

9.12 Berlekamp-Massey

Finds shortest linear recurrence from first $2L$ terms.

Returns coefficients $c_1, \dots, c_L$ where $a_n = \sum c_i \cdot a_{n-i}$.

Pair with Kitamasa for $O(L^2 \log n)$ k-th term computation.

```

vector<int> berlekampMassey(vector<int> x) {
    int n = x.size(), L = 0, m = 0;
    ll b = 1;
    vector<int> C(n), B(n);
    C[0] = B[0] = 1;
    for (int i = 0; m++ && i < n; i++) {
        ll d = x[i] % MOD;
        for (int j = 1; j <= L; j++)
            d = (d + (ll)C[j] * x[i - j]) % MOD;
        if (!d) continue;
        vector<int> T = C;
        ll c = d * modpow(b, MOD - 2) % MOD;
        for (int j = m; j < n; j++)
            C[j] = (C[j] - c * B[j - m] + MOD) % MOD;
        if (2 * L <= i) {
            L = i - L + 1; B = T;
            b = d; m = 0;
        }
    }
    C.resize(L + 1);
    C.erase(C.begin());
    for (auto &i : C) i = (MOD - i) % MOD;
    return C;
}
// Kitamasa: compute k-th term of linear recurrence
// using C (from BM) and first terms. Complexity  $O(L^2 \log k)$ .

```

9.13 Wavelet Tree

Range order statistics on arrays. Build $O(N \log M)$, query $O(\log M)$ where M = distinct values. Compresses values internally. 0-indexed. kth(l,r,k) returns k-th smallest (0-based) in $[l, r]$. count_leq(l,r,x) counts elements $\leq x$ (original value). count_seg(l,r,x,y) counts in $[x, y]$.

```

struct WT {
    int lo, hi;
    vector<int> b;
    WT *l=0, *r=0;
    WT(vector<int> &v, int lo, int hi): lo(lo), hi(hi) {
        if (v.empty() || lo+1==hi) return;
        int mid=(lo+hi)/2;
        b.resize(v.size()+1);
        for (int i=0; i<(int)v.size(); i++)
            b[i+1]=b[i]+(v[i]<mid);
        auto p=stable_partition(v.begin(), v.end(),
            [&](int x){return x<mid;})-v.begin();
    }
}

```

```

vector<int> vl(v.begin(), v.begin()+p);
vector<int> vr(v.begin()+p, v.end());
l=new WT(vl, lo, mid);
r=new WT(vr, mid, hi);
}
int kth(int k, int L, int R) {
    if (lo+1==hi) return lo;
    int left=b[R+1]-b[L];
    if (k<left) return l->kth(k, b[L], b[R+1]-1);
    else return r->kth(k-left, L-b[L], R-b[R+1]);
}
int count_leq(int L, int R, int x) {
    if (L>R || x<lo) return 0;
    if (x>=hi-1) return R-L+1;
    if (lo+1==hi) return 0;
    int mid=(lo+hi)/2;
    if (x<mid) return l->count_leq(b[L], b[R+1]-1, x);
    else return (b[R+1]-b[L]) +
        r->count_leq(L-b[L], R-b[R+1], x);
}
int count_seg(int L, int R, int x, int y) {
    if (x>y) return 0;
    return count_leq(L, R, y) - count_leq(L, R, x-1);
}
};

```

9.14 Rope (GNU)

__gnu_cxx::rope = built-in treap (split/merge/insert/erase $O(\log N)$). crope for char, rope<int> for ints. Include: #include <ext/rope>.

```

using namespace __gnu_cxx;
crope r;
r.push_back('a'); // append
r.insert(0, "hello "); // insert string at
↪ position
r.insert(5, "world"); // insert string at
↪ position
r.erase(0, 6); // erase 6 chars from
↪ position 0
r.substr(0, 5); // extract substring [0, 5)
char c = r[3]; // access  $O(\log N)$ 
cout << r << "\n"; // print whole rope
// assign from string:
string s="test"; crope r2(s.begin(), s.end());

```